



科学研究的人工智能方法学

复旦大学人工智能创新与产业研究院研究员
徐盈辉

未来25年，这125个问题将让科学家“大伤脑筋”

这125个问题，其中涉及生命科学的问题占46%，关系宇宙和地球的问题占16%，与物质科学相关的问题占14%以上，认知科学问题占9%。其余问题分别涉及数学与计算机科学、政治与经济、能源、环境和人口等。

一场正在发生的科研范式变革



《 Science 》刊登麻省理工
AI药物逆合成设计论文，AI合
成了**15个**药物分子

2019.3

利物浦大学使用AI发现**4种**可
传导锂的新固态材料，可提
供更长续航能力

2021.5

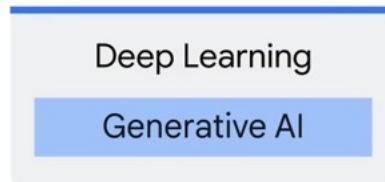
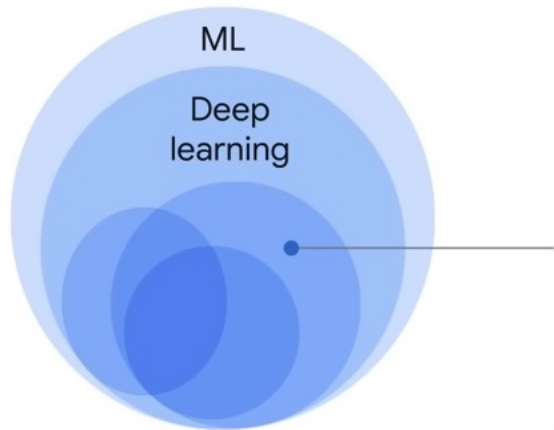
Deepmind宣布开放包括多
个动物、植物的**2亿**个蛋白质
结构的预测结果

2022.7

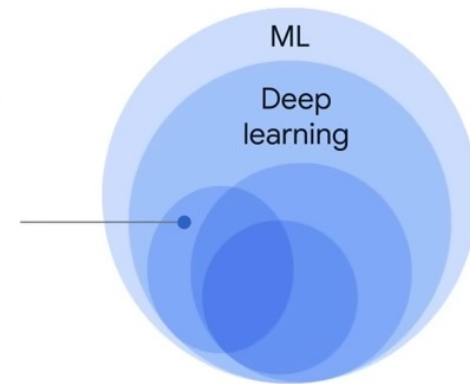
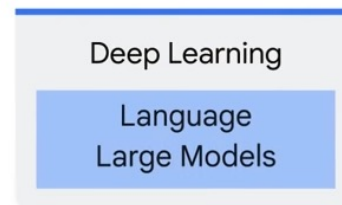


基础概念

Generative AI
is a **subset of**
Deep Learning



Large Language
Models (LLMs)
are also a **subset**
of Deep Learning



Deep Learning
Model Types



Discriminative

- Used to classify or predict
- Typically trained on a dataset of labeled data
- Learns the relationship between the features of the data points and the labels



Generative

- Generates new data that is similar to data it was trained on
- Understands distribution of data and how likely a given example is
- Predict next word in a sequence

Supervised vs unsupervised learning

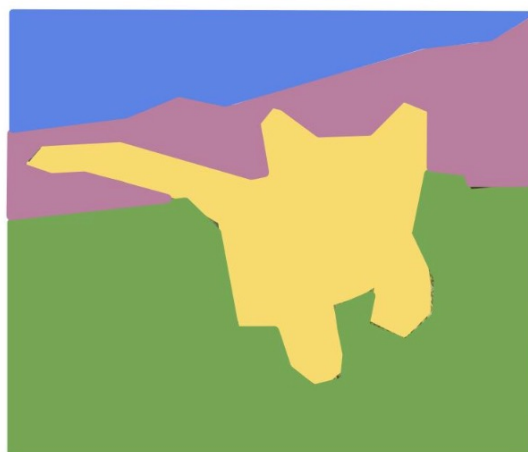
- Supervised Learning

Data: (x, y)

x is data, y is label

Goal: Learn a function to map $x \rightarrow y$

Examples: Classification, regression, object detection, semantic segmentation, image captioning, etc.



GRASS, CAT, TREE,
SKY

Semantic Segmentation

Supervised vs unsupervised learning

- Unsupervised Learning

Data: x

Just data, no labels!

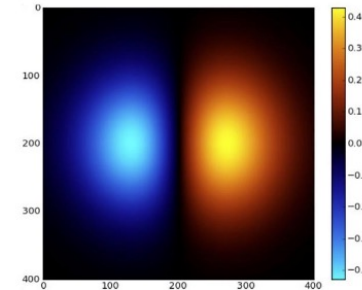
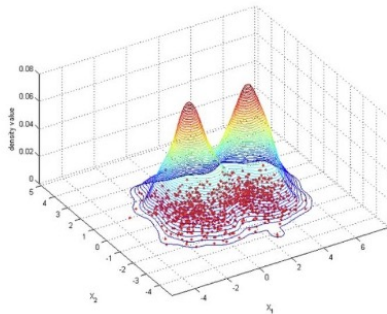
Goal: Learn some underlying hidden structure of the data

Examples: Clustering, dimensionality reduction, density estimation, etc.



Figure copyright Ian Goodfellow, 2016. Reproduced with permission.

1-d density estimation



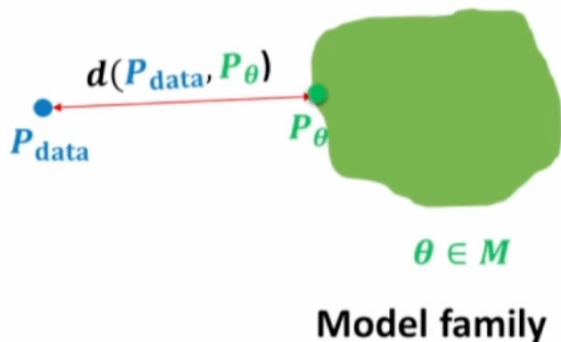
2-d density estimation

Modeling $p(x)$

2-d density images [left](#) and [right](#) are [CC0 public domain](#)



$$x_i \sim P_{\text{data}} \\ i = 1, 2, \dots, n$$



输入样本

生成样本

训练数据 $\sim P_{\text{data}}(x)$

生成样本 $\sim p_{\text{model}}(x)$

我们如何来学习

$p_{\text{data}}(x) \sim p_{\text{model}}(x)$

我们的目标是学习一个概率分布 $p(x)$:

1.生成 (Generation) : 如果从 $p(x)$ 中抽样一个新数据 x_{new} , 那么这个样本应该看起来像一只狗.

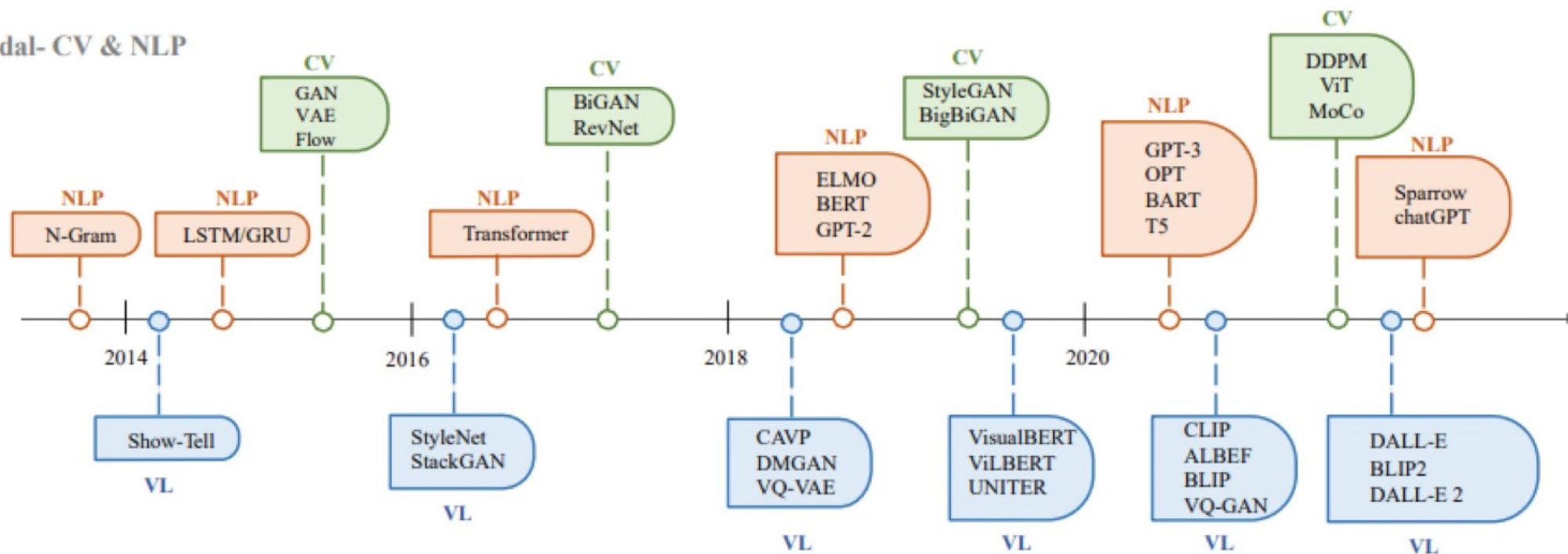
2.密度估计 (Density Estimation) : 对于一个看起来像狗的样本, $p(x)$ 的值应较高, 而对于其他不符合的数据 $p(x)$ 应较低

3.无监督表示学习 (Unsupervised Representation Learning) : 模型能够从数据中自动学习到类别的潜在特征, 而无需监督标签

生成式AI类型

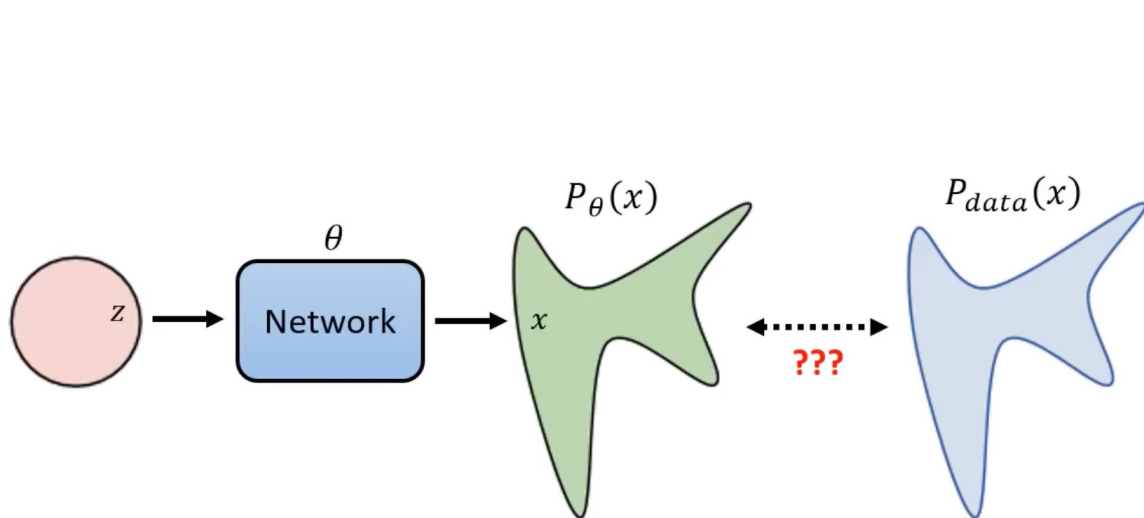
- Generative AI learns from existing data and generate entirely new content in various forms, such as text, images, music, and videos, with similar characteristics to the original data.
- Scaling Law + Self-supervised Learning
- Autoregressive Generative Models + Diffusion Generative Models

Unimodal- CV & NLP



生成式AI的一些数学基础

极大似然估计 (MLE)



$$\theta^* = \arg \max_{\theta} L$$

$$= \arg \max_{\theta} \prod_{i=1}^m P_G(x^i; \theta)$$



$$\arg \max_{\theta} \log \prod_{i=1}^m P_G(x^i; \theta)$$

$$= \arg \max_{\theta} \sum_{i=1}^m \log P_G(x^i; \theta)$$

$$\approx \arg \max_{\theta} E_{x \sim P_{data}} [\log P_G(x; \theta)]$$

$$= \arg \max_{\theta} \int_x P_{data}(x) \log P_G(x; \theta) dx$$

$$= \arg \max_{\theta} \int_x P_{data}(x) \log P_G(x; \theta) dx - \int_x P_{data}(x) \log P_{data}(x) dx$$

$$= \arg \min_{\theta} \underline{KL(P_{data}(x) || P_G(x; \theta))}$$

p_{data} & P_{θ} 之间的差异

1. 从 $P_{data}(x)$ 采样 m 个样本 $\{x^1, x^2, \dots, x^m\}$
2. 计算采样样本的似然函数 $L = \prod_{i=1}^m P_G(x^i; \theta)$
3. 计算使得似然函数 L 最大的参数 θ : $\theta^* = \arg \max_{\theta} L = \arg \max_{\theta} \prod_{i=1}^m P_G(x^i; \theta)$

找到 θ 使得 $P_G(x; \theta)$ 与目标分布 $P_{data}(x)$ 的KL散度尽可能低, 也就是使得两者的分布尽可能接近, 实现用确定的分布 $P_G(x; \theta)$ 极大似然 $P_{data}(x)$

Kullback-Leibler divergence (MLE)

$$D_{KL}(p||q) = \int_x p(x) \log \frac{p(x)}{q(x)} dx$$

$p(x)$: 真实分布

$q(x)$: 近似分布

$$D_{KL}(P||Q) \neq D_{KL}(Q||P)$$

定义： 衡量两个概率分布之间差异的非对称指标

物理学： 信息损失或“额外的熵”，即从一个分布P（真实分布）到另一个分布Q（近似分布）所需的额外信息量

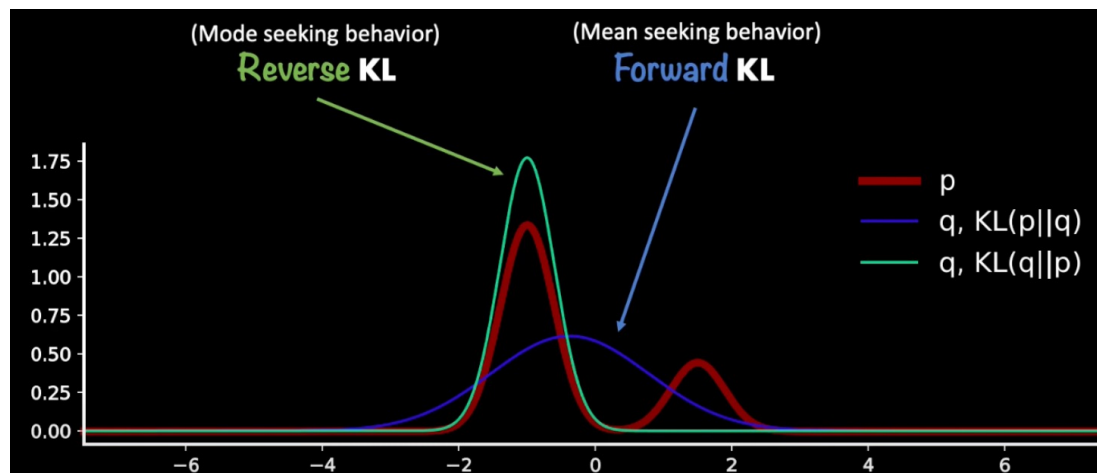
KL-divergence通常用来量化模型学习的效率和误差，尤其是在变分推断和信息论中

前向KL散度 (Forward KL) :

均值寻求行为 (Mean Seeking Behavior)

反向KL散度 (Reverse KL)

模式寻求行为 (Mode Seeking Behavior)





Jensen-Shannon Divergence （ JS ）

$$D_{JS}(p||q) = \frac{1}{2}D_{KL}(p||\frac{p+q}{2}) + \frac{1}{2}D_{KL}(q||\frac{p+q}{2})$$

- 1. **对称性**：Jensen-Shannon散度是对称的，即 $D_{JS}(p||q) = D_{JS}(q||p)$ 。
- 2. **有限值域**：JSD的值介于0到1之间（当使用对数底为2时）。若 $p = q$ ，则 $D_{JS}(p||q) = 0$ ；若 p 和 q 完全不重叠，JSD接近1。
- 3. **平滑性**：JSD能够处理KL散度无法处理的情况，比如 q 或 p 为零的情况，避免了无限大的问题。

特性	KL散度 ($D_{KL}(P Q)$)	JS散度 ($D_{JS}(P Q)$)
公式	$D_{KL}(P Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$	$D_{JS}(P Q) = \frac{1}{2}D_{KL}(P M) + \frac{1}{2}D_{KL}(Q M)$ 其中 $M = \frac{P+Q}{2}$
非对称性	非对称: $D_{KL}(P Q) \neq D_{KL}(Q P)$	对称: $D_{JS}(P Q) = D_{JS}(Q P)$
值域	$[0, \infty]$ ，如果 Q 与 P 的支持不同，可能会趋向无穷大	取值在 $[0, 1]$ （对数底为2时），解决了KL散度中无穷大的问题
行为	惩罚未覆盖区域：倾向于覆盖所有真实分布的高概率区域（均值寻求或模式覆盖）	更平滑地衡量分布之间的差异，具有对称性，且不容易受到低概率区域影响
应用场景	用于优化问题或需要有方向性比较的场景，如变分推断	用于更加稳健和对称的分布比较场景，如衡量生成分布和真实分布之间的相似性



高斯函数空间的特性

高斯函数（Gaussian function）在数学、统计和信号处理中具有许多重要的特性

$$f(x) = A \exp \left(-\frac{(x - \mu)^2}{2\sigma^2} \right)$$

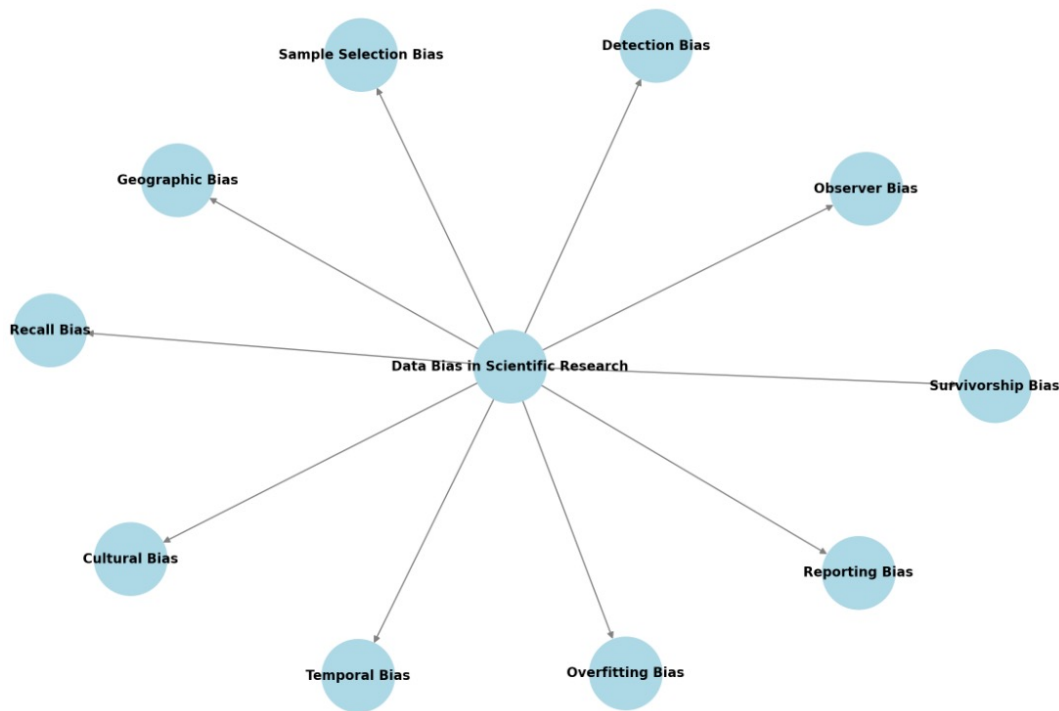
- **相乘**：两个高斯函数的乘积仍为高斯函数，但均值和方差变化。
- **相除**：高斯函数的商一般不再是高斯函数，除非均值和方差一致。
- **相加**：高斯函数的和通常表现为高斯混合分布。
- **相减**：高斯函数的差形成双峰或鞍形分布，应用于图像处理中的边缘检测。



生成式AI对于科学研究的价值： debiasing

生成式AI在科学研究中的意义之一在于“去偏差”（debiasing）

生成式模型能够识别和揭示数据集中的底层特征，这对于构建更公平、代表性的数据集至关重要



医学影像分析中数据偏差

生态学/生物多样性研究中的数据偏差

病理学数据的无偏差合成

社会科学和心理学研究中的公平数据集

金融市场分析中的公平数据构建



生成式AI对于科学研究的价值：异常检测

生成式模型能够识别和检测数据分布中的异常值，从而帮助研究者在训练模型时更好地应对罕见或极端的情况。

方法	描述	示例
概率估计	利用VAE、GAN等生成模型，检测低概率数据点	自动驾驶中识别少见天气
重构误差	自编码器无法准确重构异常数据点	自动驾驶中检测恶劣天气
潜在空间分析	异常点在潜在空间中离中心较远	人脸模型检测特殊光照
密度估计	利用正态化流或高斯混合模型检测低密度点	识别金融市场异常波动
GAN判别器	判别器检测不符合正常模式的数据	健康细胞检测异常细胞
自监督学习	高预测误差表示异常	监控中检测不寻常行为
能量模型	高能量值标记异常	工业设备异常振动检测

- 提升泛化能力：包含异常样本的模型对罕见情况有更好的适应性
- 降低不可预测性：提前检测异常，减少模型的风险
- 多行业应用：适用于自动驾驶、医学、金融等需要鲁棒性的领域

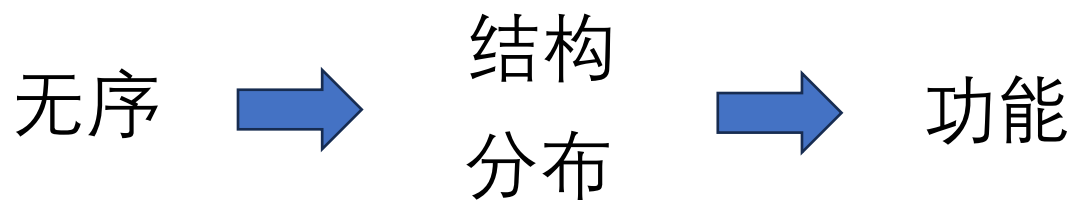


生成式AI的意义

消除数据偏见的工具

生成式模型“能够揭示数据集中的潜在特征”，这意味着生成式模型可以识别并生成原始数据中可能被低估或缺失的多样性特征。

开放域适配能力

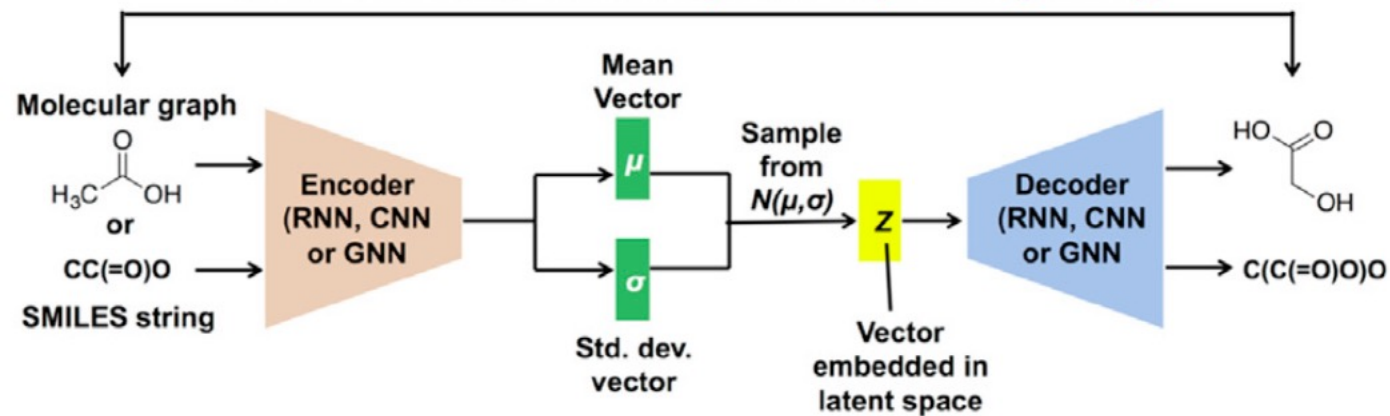




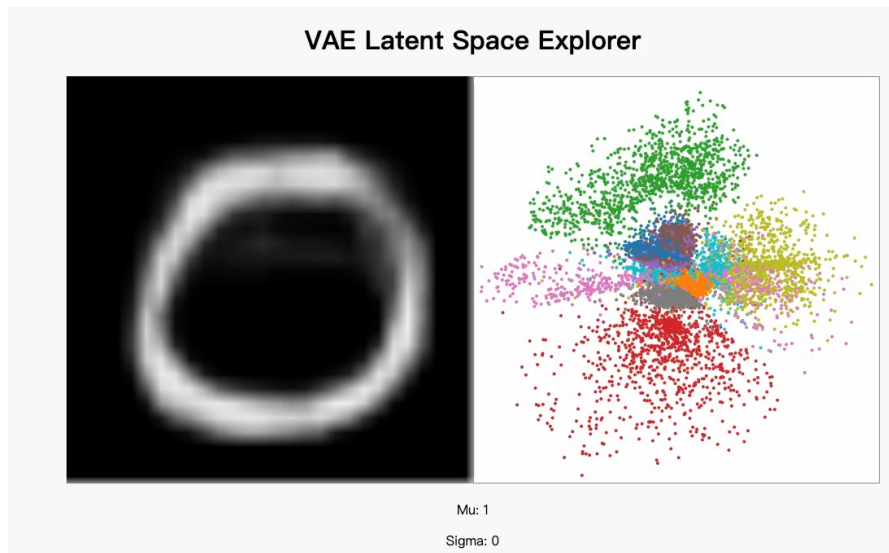
经典的生成式AI方法



Minimize difference between input and output during training



1. 潜在空间捕捉了数据的核心特征
2. 相邻点表示相似的数据
3. 解码器根据潜在特征生成数据



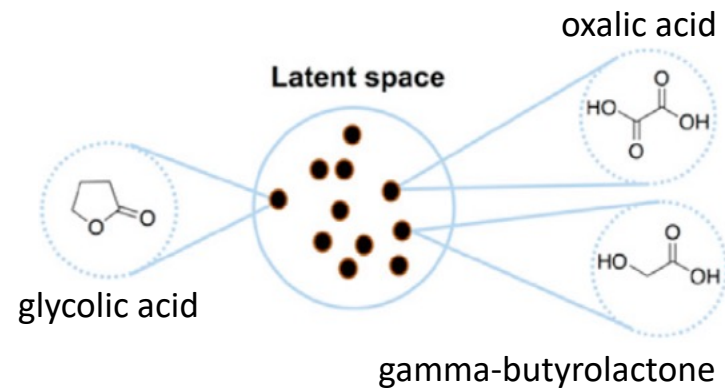
变分自编码器

损失函数

$$\mathcal{L}_{VAE} = \underbrace{\mathbb{E}_{q(z|x)}[-\log p(x|z)]}_{\text{重构损失}} + \underbrace{D_{KL}(q(z|x) \parallel p(z))}_{\text{KL散度}}$$

作用

1. 能进行数据的降维和特征提取
2. 能用于生成新的类似样本



VAE

GANs

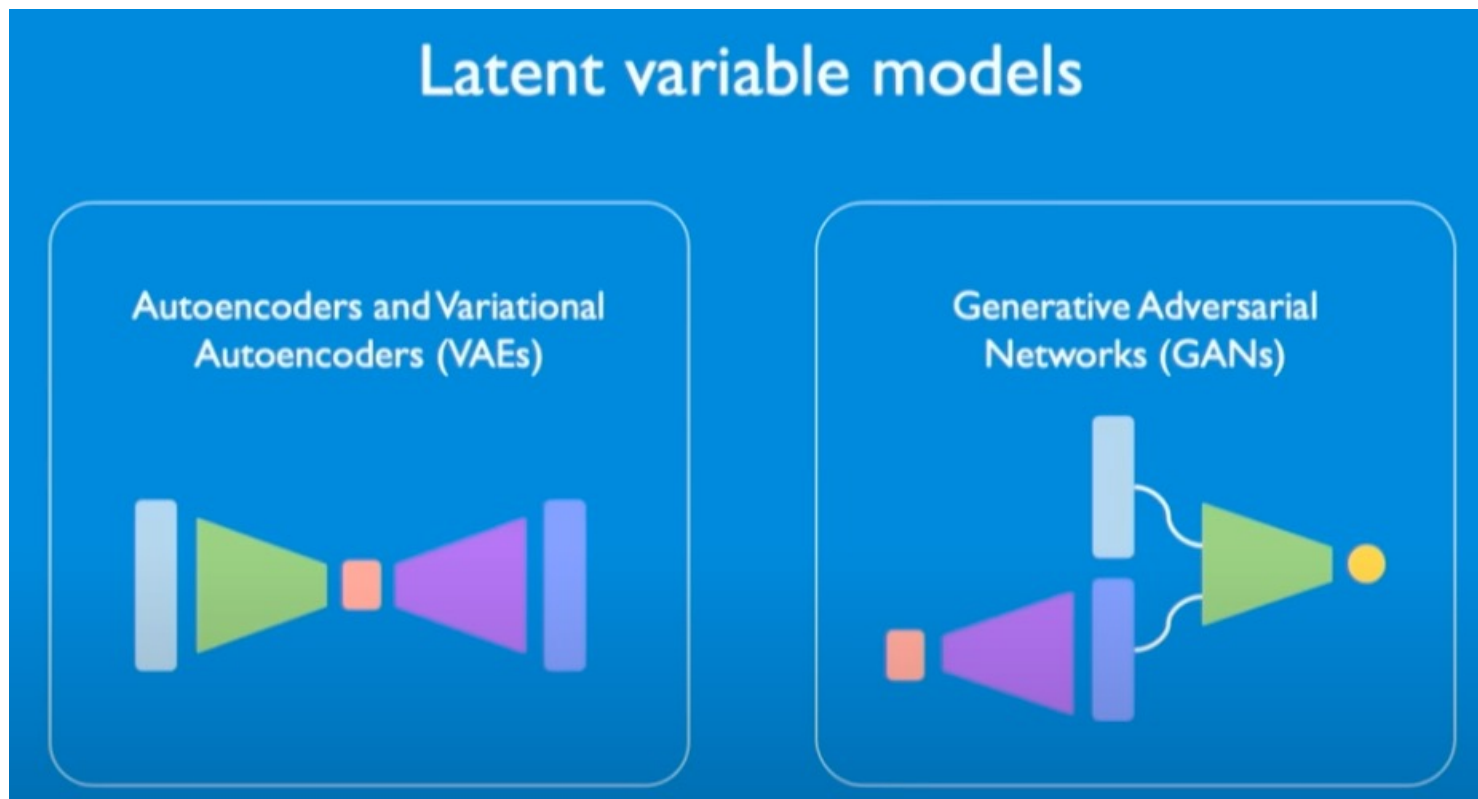
AR

Diffusion Models

Flow Based

- Variational Autoencoders (VAEs) (Kingma and Welling 2013)
- Generative Adversarial Networks (GANs) (Goodfellow *et al.* 2014)
- Flow-based Generative Models (Rezende and Mohamed 2015)
- Autoregressive Models (Vaswani *et al.* 2017)
- Diffusion Models (DDPMs) (Ho, Jain and Abbeel 2020)
- Score Based Generative Models (Song and Ermon 2019, Song *et al.* 2021)

隐变量模型 (latent variable models)



自编码器与变分自编码器

生成对抗网络

定义： 隐藏的、不可直接观测的变量，它代表数据的底层结构或特征。

隐变量

潜变量是数据生成的真实解释因素，但我们无法直接观测到它们，只能通过观测到的数据（影子）来推测其存在和特性



柏拉图的洞穴

不可直接观测

定义： 潜变量是数据背后那些不可直接观测但决定数据分布的隐藏因素。

物理学意义： 潜变量可以理解为那些不可直接测量的物理属性或内在特征

量子力学中的波函数，热力学和统计物理中的微观状态，相变中的序参数

数学上的表示： 潜变量可以用来描述观测数据的生成过程。假设 x 是观测变量， z 是潜变量

$$p(x) = \int p(x|z)p(z) dz$$

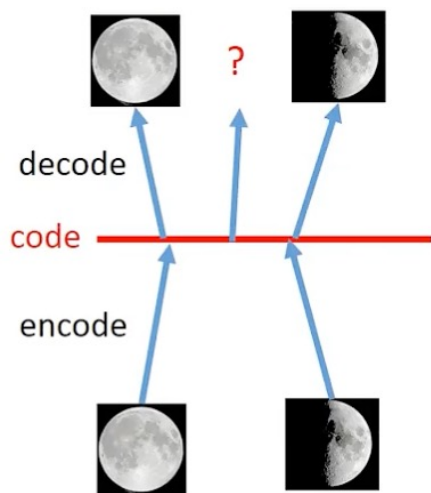
每一个样本都是高维度空间的一个点，生成式 a_i 就是从大量的训练样本在高维空间所构成的population，去尝试学习到样本空间的概率密度函数；

当我们知道了样本空间的概率密度函数 $P(x)$ ，我们就可以通过采样和重构的方法来生成样本。

编码器： 编码器将输入数据 x 映射到潜在空间 z

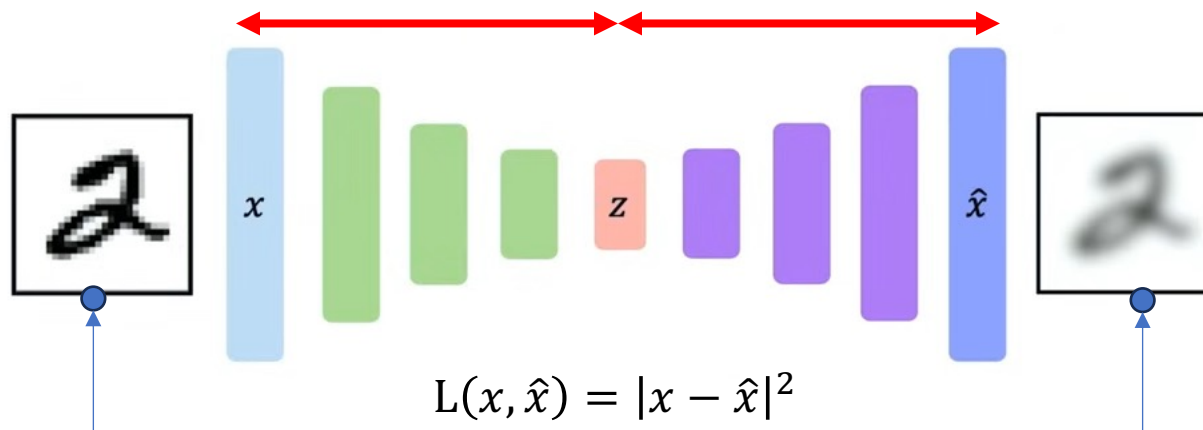
解码器： 解码器将潜在表示 z 映射回重构的数据 $\hat{x}$

直观的解释



$$z = f(x; W, b)$$

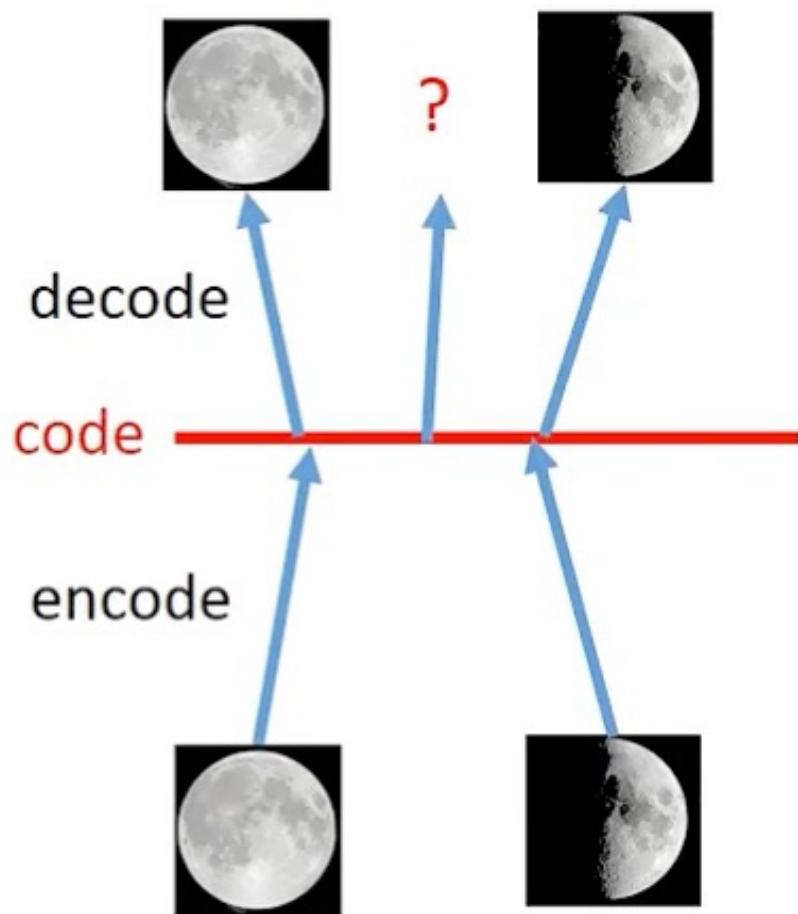
$$\hat{x} = g(z; W', b')$$



$$L(\mathbf{x}, \hat{\mathbf{x}}) = - \sum_i (x_i \log(\hat{x}_i) + (1 - x_i) \log(1 - \hat{x}_i))$$

重构误差

Auto Encoder 的局限性 @ 生成式AI



- 缺乏生成能力：无法控制数据生成的多样性。
- 潜变量分布无约束：生成数据时潜变量分布无结构。
- 难以建模复杂数据分布：在多模态数据上表现不佳。
- 潜变量缺乏可解释性：潜变量不具有明确的物理或语义意义。
- 对噪声敏感：训练数据中的异常值会影响潜变量表征的质量。
- 泛化能力有限：在分布变化的数据上难以泛化。
- 维度选择的困难：潜变量空间的维度影响重构和压缩效果。



Diederik P. Kingma



arXiv:1312.6114v11 [stat.ML] 10 Dec 2022

Auto-Encoding Variational Bayes

Diederik P. Kingma
Machine Learning Group
Universiteit van Amsterdam
dpkingma@gmail.com

Max Welling
Machine Learning Group
Universiteit van Amsterdam
welling.max@gmail.com

Abstract

How can we perform efficient inference and learning in directed probabilistic models, in the presence of continuous latent variables with intractable posterior distributions, and large datasets? We introduce a stochastic variational inference and learning algorithm that scales to large datasets and, under some mild differentiability conditions, even works in the intractable case. Our contributions are two-fold. First, we show that a reparameterization of the variational lower bound yields a lower bound estimator that can be straightforwardly optimized using standard stochastic gradient methods. Second, we show that for i.i.d. datasets with continuous latent variables per datapoint, posterior inference can be made especially efficient by fitting an approximate inference model (also called a recognition model) to the intractable posterior using the proposed lower bound estimator. Theoretical advantages are reflected in experimental results.

1 Introduction

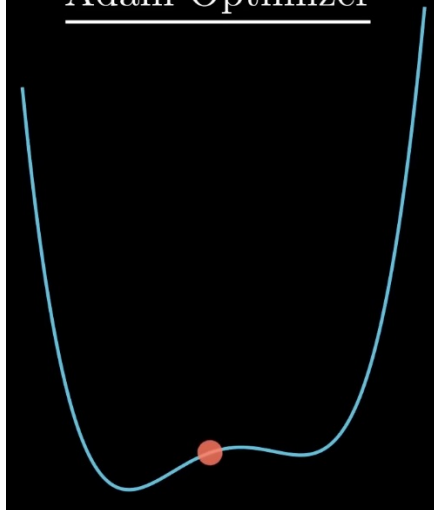
How can we perform efficient approximate inference and learning with directed probabilistic models whose continuous latent variables and/or parameters have intractable posterior distributions? The variational Bayesian (VB) approach involves the optimization of an approximation to the intractable posterior. Unfortunately, the common mean-field approach requires analytical solutions of expectations w.r.t. the approximate posterior, which are also intractable in the general case. We show how a reparameterization of the variational lower bound yields a simple differentiable unbiased estimator of the lower bound; this SGVB (Stochastic Gradient Variational Bayes) estimator can be used for efficient approximate posterior inference in almost any model with continuous latent variables and/or parameters, and is straightforward to optimize using standard stochastic gradient ascent techniques.

For the case of an i.i.d. dataset and continuous latent variables per datapoint, we propose the Auto-Encoding VB (AEVB) algorithm. In the AEVB algorithm we make inference and learning especially efficient by using the SGVB estimator to optimize a recognition model that allows us to perform very efficient approximate posterior inference using simple ancestral sampling, which in turn allows us to efficiently learn the model parameters, without the need of expensive iterative inference schemes (such as MCMC) per datapoint. The learned approximate posterior inference model can also be used for a host of tasks such as recognition, denoising, representation and visualization purposes. When a neural network is used for the recognition model, we arrive at the *variational auto-encoder*.

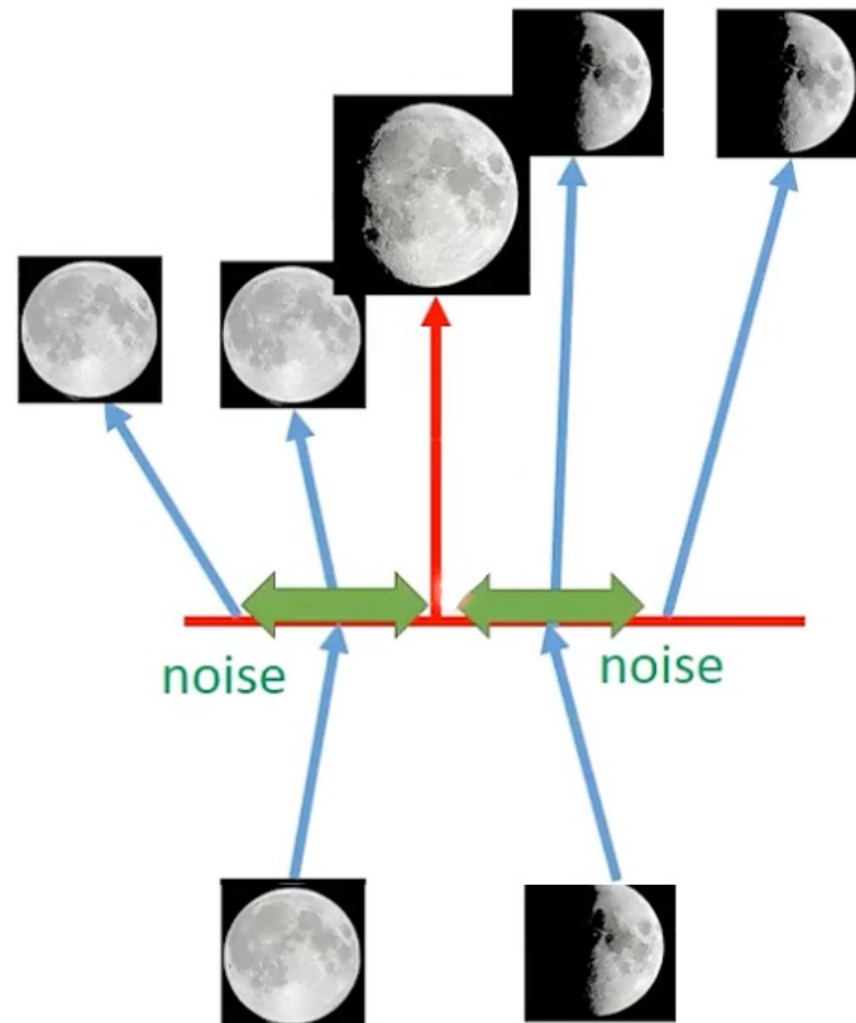
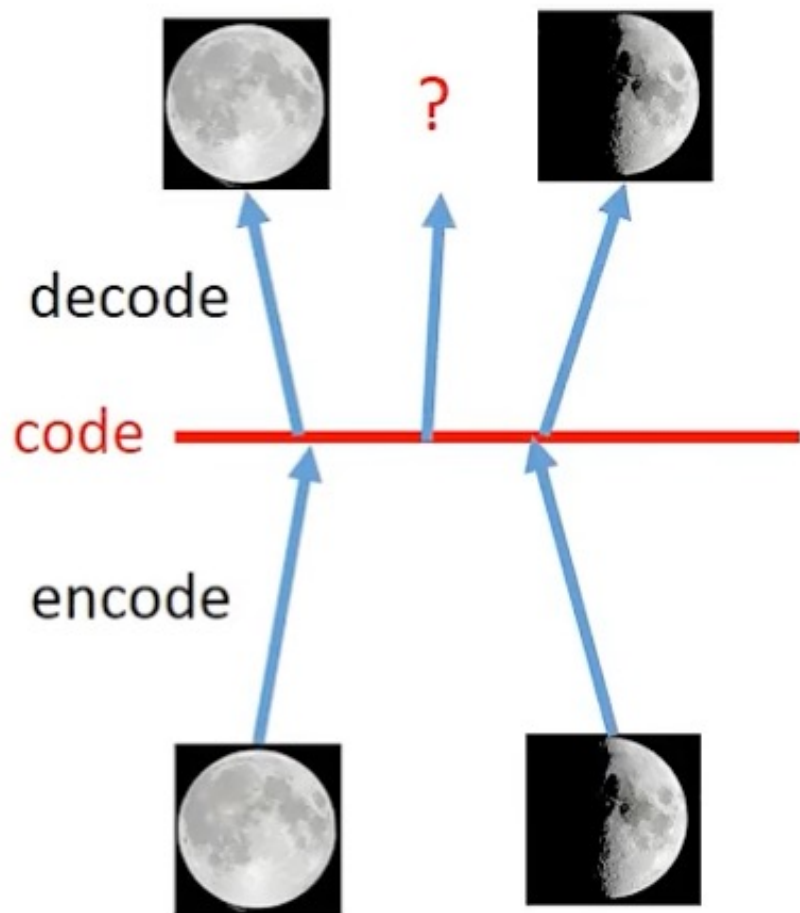
2 Method

The strategy in this section can be used to derive a lower bound estimator (a stochastic objective function) for a variety of directed graphical models with continuous latent variables. We will restrict ourselves here to the common case where we have an i.i.d. dataset with latent variables per datapoint, and where we like to perform maximum likelihood (ML) or maximum a posteriori (MAP) inference on the (global) parameters, and variational inference on the latent variables. It is, for example,

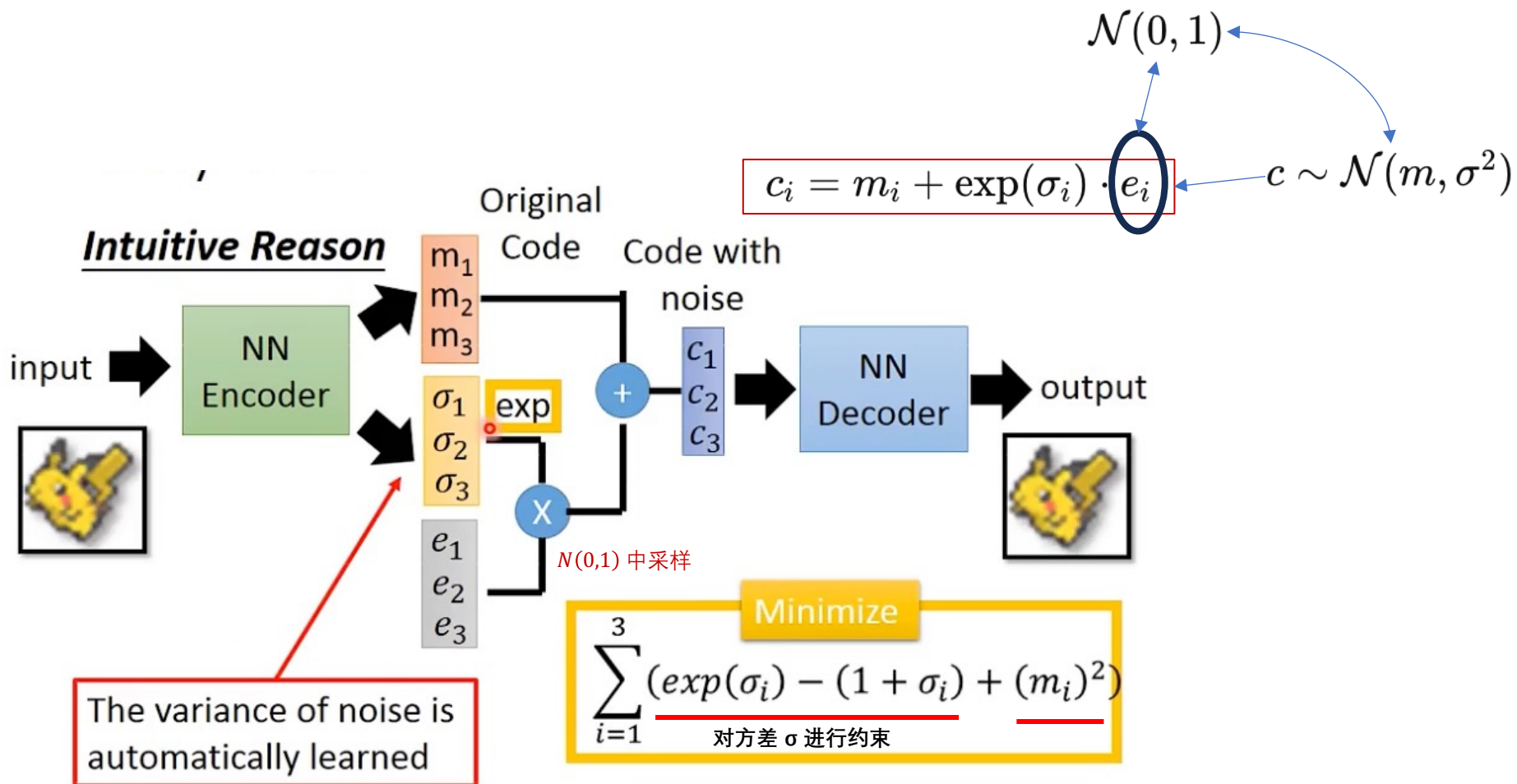
Adam Optimizer



变分自编码器 (VAE)



变分自编码器 (VAE) 的直观解释



Gaussian mixture model --> VAE

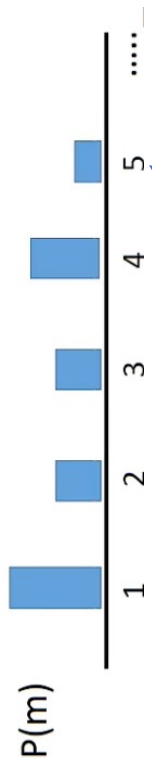
How to sample?

$m \sim P(m)$ (multinomial)

m is an integer

$x|m \sim N(\mu^m, \Sigma^m)$

$$P(x) = \sum_m P(m)P(x|m)$$



$a_0, b_0, \dots, h_0$
 $a_1, b_1, \dots, h_1$
 $\dots$
 $a_k, b_k, \dots, h_k$

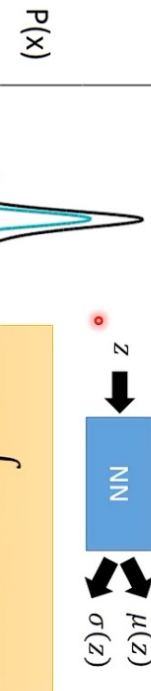
样本表征有n个 attributes, 每个 attribute 在样本空间是符合高斯分布

$$P(x) = \int_z P(z)P(x|z)dz$$

$$L = \sum_x \log P(x)$$

Maximizing the likelihood of the observed x

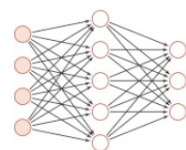
Even though z is from $N(0, I)$, $P(x)$ can be very complex



$$P(x) = \int P(z)P(x|z)dz$$

Infinite Gaussian

$x|z \sim N(\mu(z), \sigma(z))$
 $\mu(z), \sigma(z)$ is going to be estimated



$f: z \rightarrow u$

$g: z \rightarrow \sigma$

VAE

$z \sim N(0, I)$

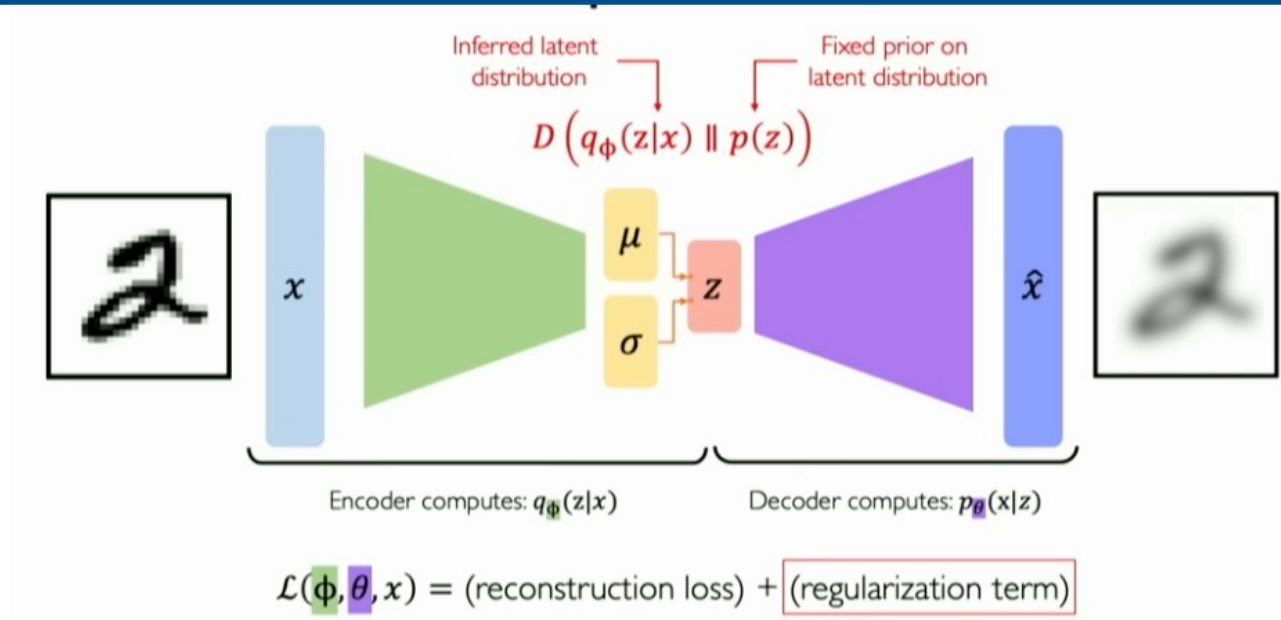
z is a vector from normal distribution

$x|z \sim N(\mu(z), \sigma(z))$

Each dimension of z represents an attribute



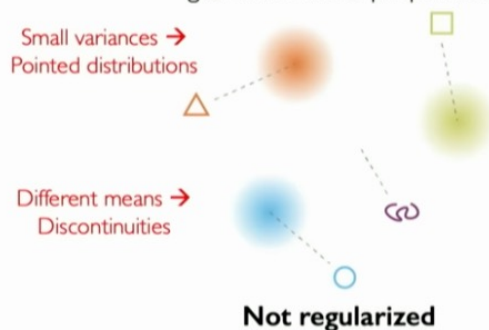
VAE 算法的示意图



Intuition on regularization and the Normal prior

1. **Continuity**: points that are close in latent space $\rightarrow$ similar content after decoding
2. **Completeness**: sampling from latent space $\rightarrow$ "meaningful" content after decoding

Encoding as a distribution does not guarantee these properties!



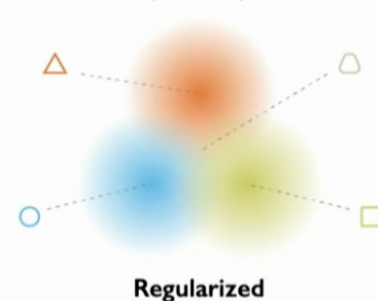
Intuition on regularization and the Normal prior

1. **Continuity**: points that are close in latent space $\rightarrow$ similar content after decoding
2. **Completeness**: sampling from latent space $\rightarrow$ "meaningful" content after decoding

Encoding as a distribution does not guarantee these properties!



Normal prior $\rightarrow$ continuity + completeness



VAE 算法原理解读

$$L = \sum_x \log P(x)$$

Maximizing the likelihood of the observed x

辅助变量

$$\log P(x) = \int_z q(z|x) \log P(x) dz$$

$q(z|x)$ can be any distribution

$$= \int_z q(z|x) \log \left(\frac{P(z, x)}{P(z|x)} \right) dz = \int_z q(z|x) \log \left(\frac{P(z, x) q(z|x)}{q(z|x) P(z|x)} \right) dz$$

$$= \int_z q(z|x) \log \left(\frac{P(z, x)}{q(z|x)} \right) dz + \int_z q(z|x) \log \left(\frac{q(z|x)}{P(z|x)} \right) dz$$

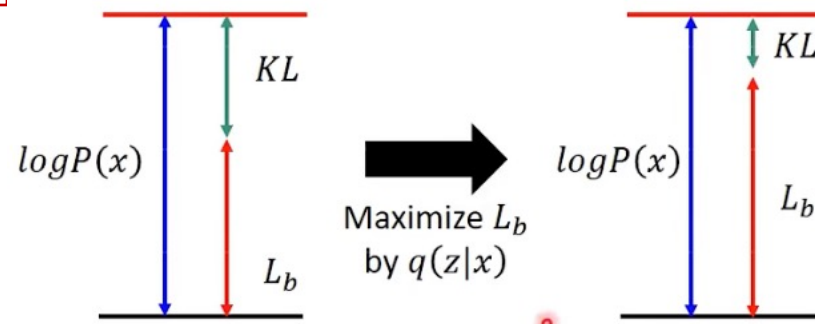
$KL(q(z|x) || P(z|x))$

≥ 0

L_b : 证据下界 (ELBO)

$$\geq \int_z q(z|x) \log \left(\frac{P(x|z)P(z)}{q(z|x)} \right) dz$$

我们通过最大化 L_b 来让 $q(z|x)$ 尽量接近 $P(z|x)$, 以实现间接优化的效果



对抗



VAE 算法原理解读

Maximize
$$L_b = \int_z q(z|x) \log \left(\frac{P(x|z)P(z)}{q(z|x)} \right) dz$$

$$= \underbrace{\int_z q(z|x) \log \left(\frac{P(z)}{q(z|x)} \right) dz}_{-KL(q(z|x)||P(z))} + \underbrace{\int_z q(z|x) \log P(x|z) dz}$$

Minimize $KL(q(z|x)||P(z))$

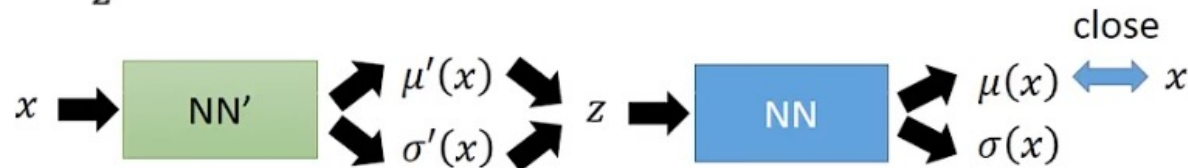
$$z|x \sim N(\mu'(x), \sigma'(x))$$



Minimize
$$\sum_{i=1}^3 (\exp(\sigma_i) - (1 + \sigma_i) + (m_i)^2)$$

Maximize

$$\int_z q(z|x) \log P(x|z) dz = E_{q(z|x)}[\log P(x|z)]$$

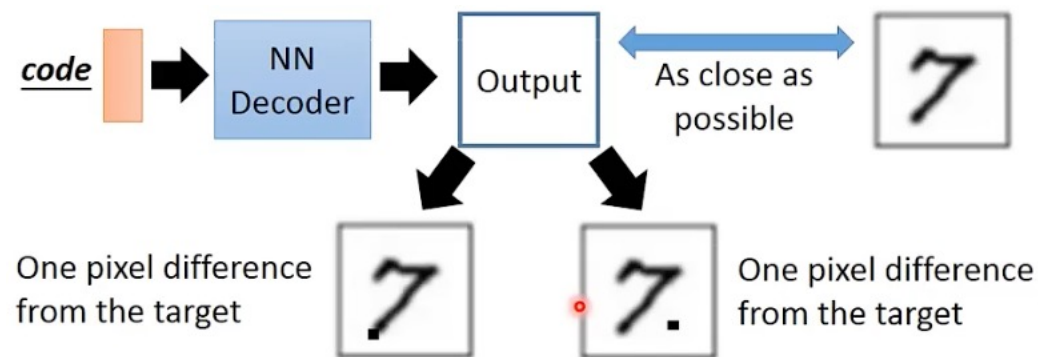


等同于 auto-encoder

VAE的损失函数设计

$$\mathcal{L}_{\text{VAE}} = \mathbb{E}_{q(z|x)}[\log P(x|z)] - D_{KL}(q(z|x)||P(z))$$

1. 潜在空间的限制
2. 解码器的局限性
3. 模型训练中的平均化效应
4. 样本多样性不足



VAE仅仅是在“记住”现有的图像，而不是去创造新的图像。

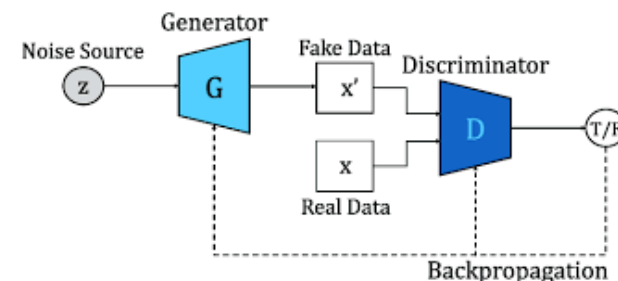
Generative Adversarial Network (GAN)

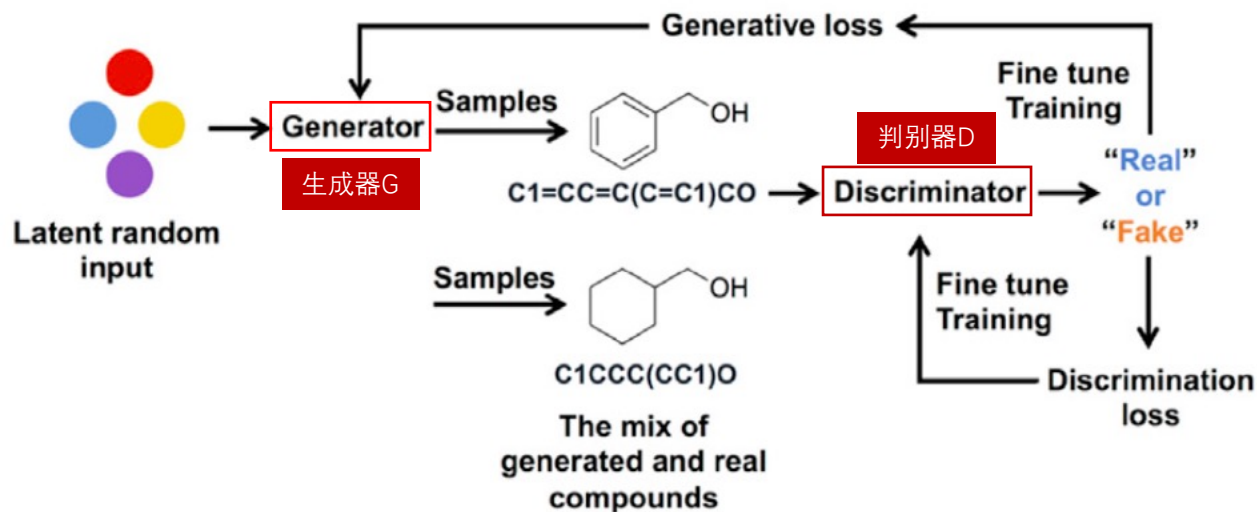
Generative Adversarial Nets

Ian J. Goodfellow, Jean Pouget-Abadie*, Mehdi Mirza, Bing Xu, David Warde-Farley,
Sherjil Ozair†, Aaron Courville, Yoshua Bengio‡
Département d'informatique et de recherche opérationnelle
Université de Montréal
Montréal, QC H3C 3J7

Abstract

We propose a new framework for estimating generative models via an adversarial process, in which we simultaneously train two models: a generative model G that captures the data distribution, and a discriminative model D that estimates the probability that a sample came from the training data rather than G . The training procedure for G is to maximize the probability of D making a mistake. This framework corresponds to a minimax two-player game. In the space of arbitrary functions G and D , a unique solution exists, with G recovering the training data distribution and D equal to $\frac{1}{2}$ everywhere. In the case where G and D are defined by multilayer perceptrons, the entire system can be trained with backpropagation. There is no need for any Markov chains or unrolled approximate inference networks during either training or generation of samples. Experiments demonstrate the potential of the framework through qualitative and quantitative evaluation of the generated samples.





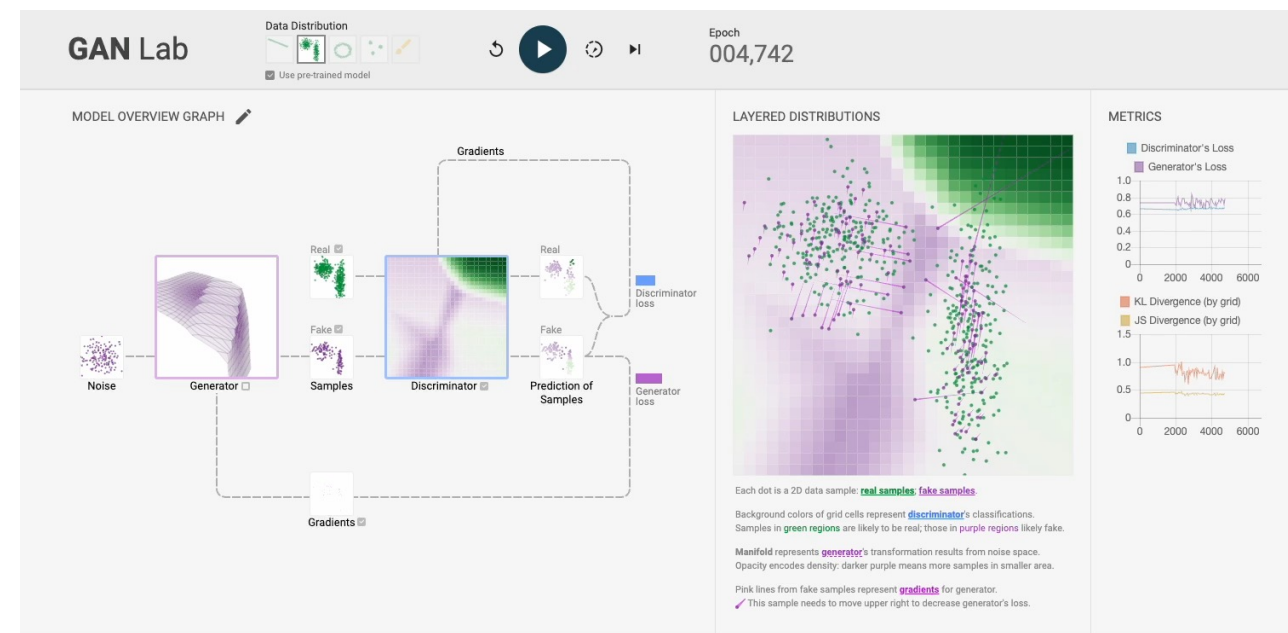
Generative Adversary Network

生成器 G: 从一个简单的随机噪声（例如高斯噪声或均匀分布）中生成逼真的样本。生成器的目标是“欺骗”判别器，使其认为生成的样本是真实的

$$\mathcal{L}_G = -\mathbb{E}_{z \sim p_z(z)} [\log D(G(z))]$$

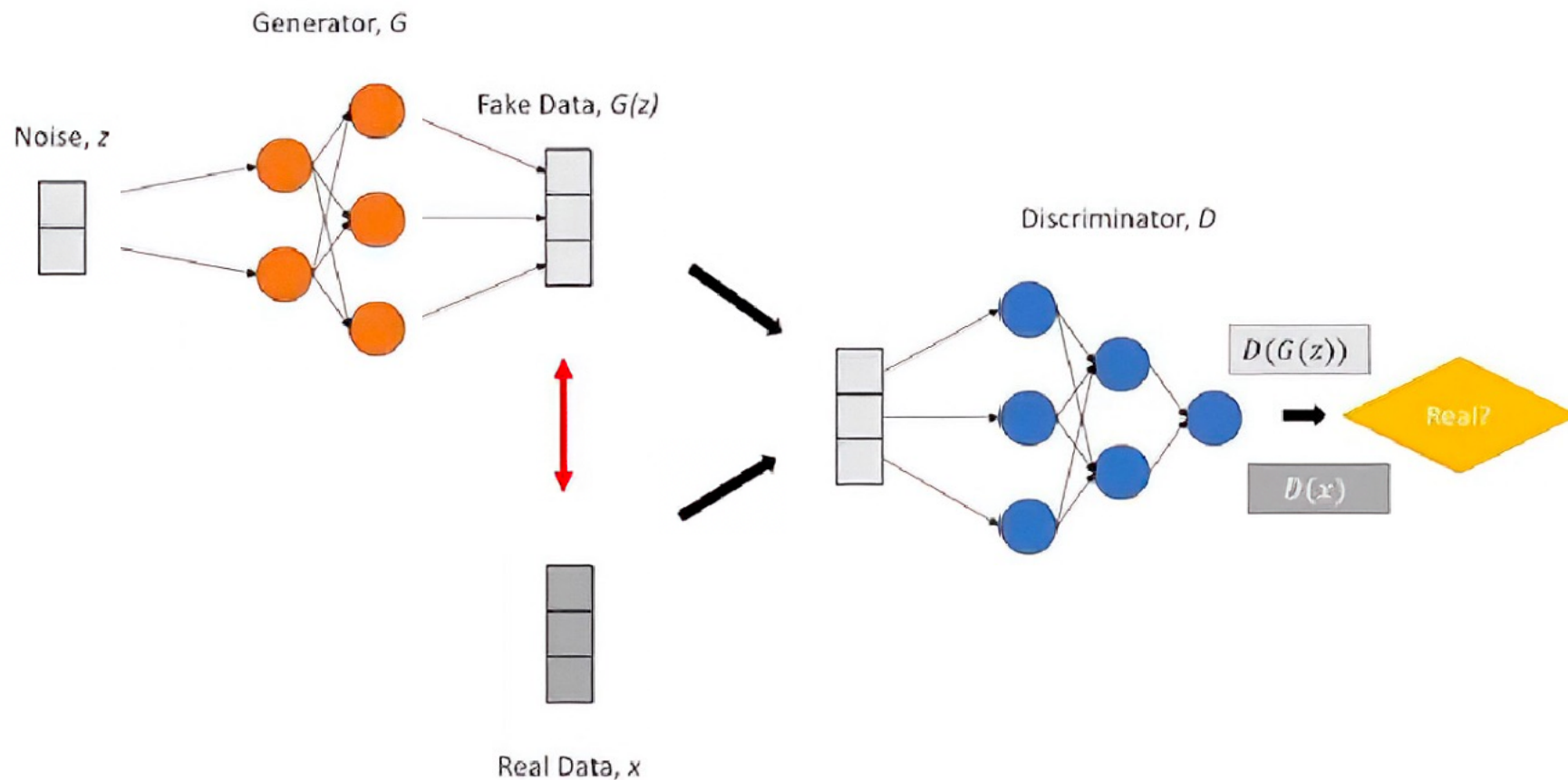
判别器 D: 区分生成的数据和真实数据。它的目标是最大限度地正确判断输入数据是真实数据还是生成的数据。

$$\mathcal{L}_D = -\mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] - \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$



GAN 的训练目标是通过**博弈论**的方式，使生成器和判别器之间相互对抗，生成器试图生成逼真的样本以欺骗判别器，而判别器试图区分真实数据和生成数据。最终，生成器生成的样本应该难以与真实数据区分开来

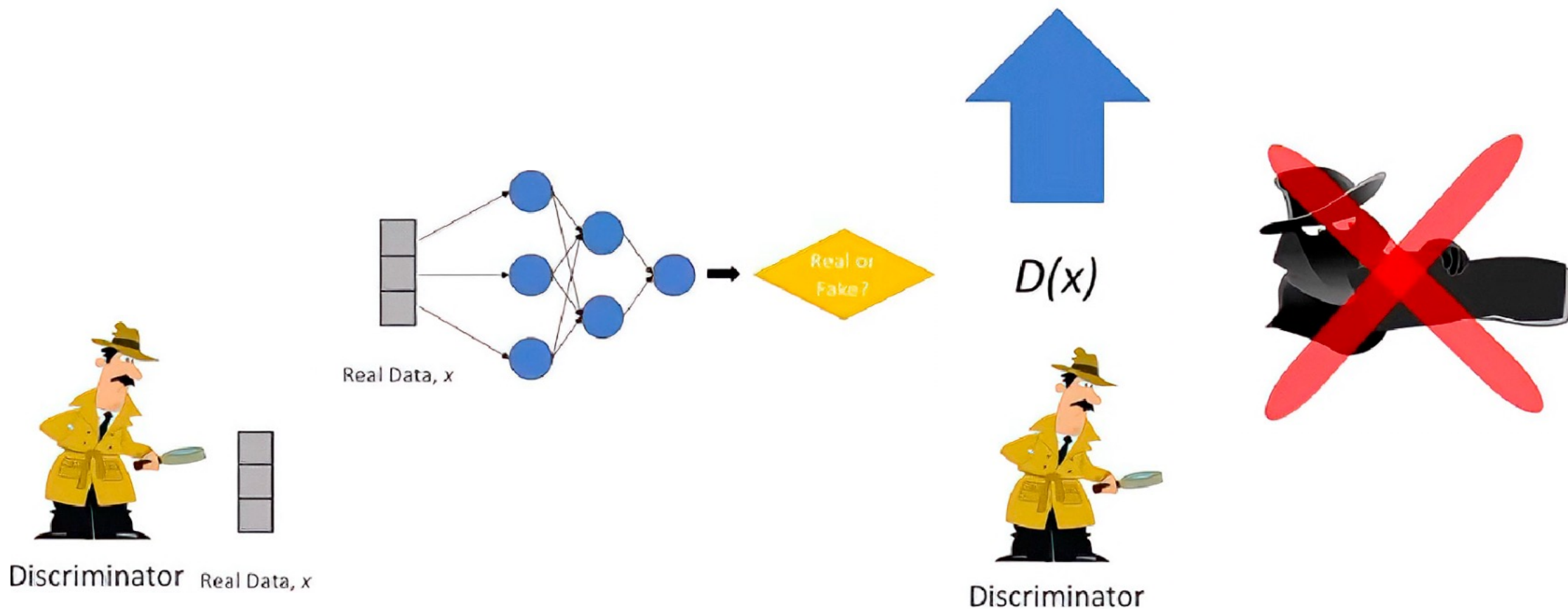
Generative Adversarial Network (GAN)





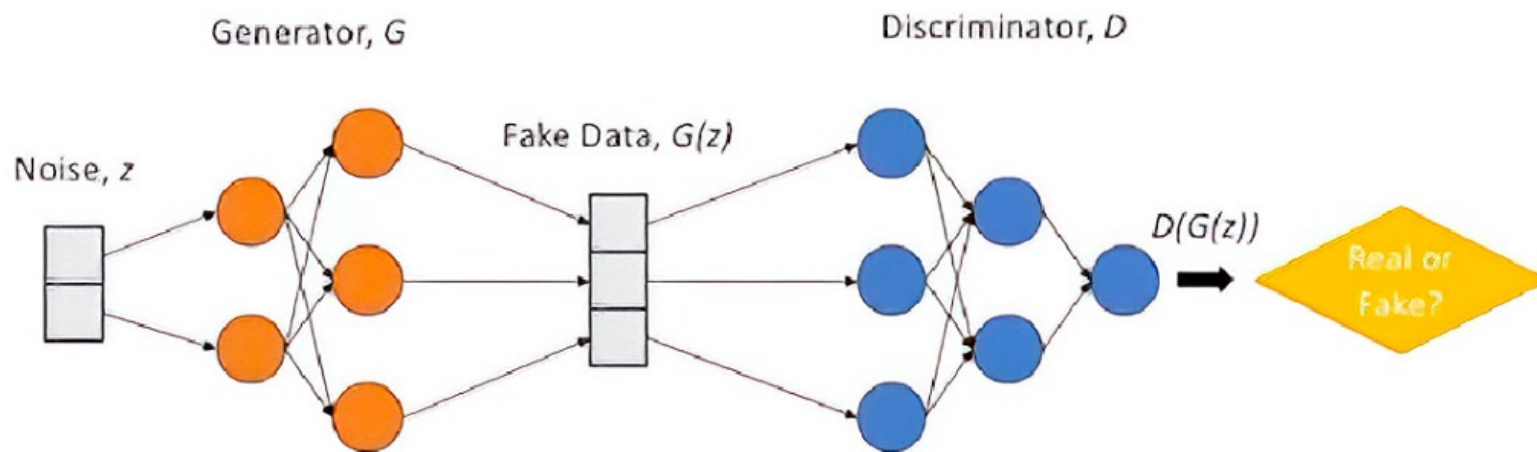
discriminator @ GAN

$$V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$



Generator @ GAN

$$V(D, G) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

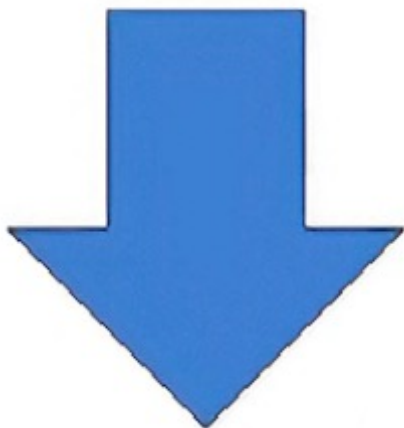
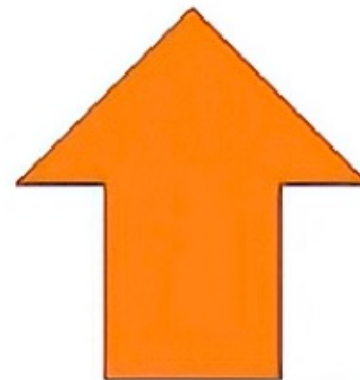


Discriminator

Discriminator



$$D(G(z))$$

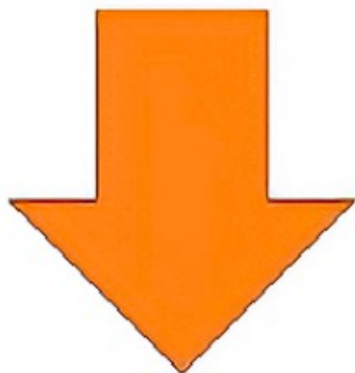
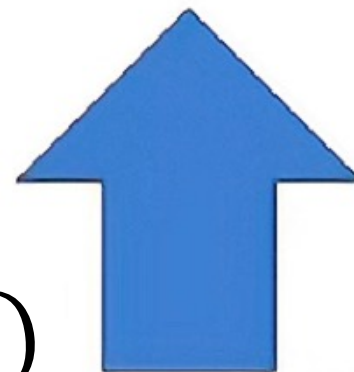


Generator

Generator



$$1 - D(G(z))$$



Discriminator

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

- **for** each training iteration **do**
 - **for** k steps **do**
 - Sample m noise samples $\{z_1, \dots, z_m\}$ and transform with Generator
 - Sample m real samples $\{x_1, \dots, x_m\}$ from real data
 - Update the Discriminator by **ascending** the gradient:

$$\uparrow \nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right]$$

- **end for**
- Sample m noise samples $\{z_1, \dots, z_m\}$ and transform with Generator
- Update the Generator by **descending** the gradient:

$$\downarrow \nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)})))$$

- **end for**





Math/theory behind GAN

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

$$V(G, D) = \int_{\mathbf{x}} p_{\text{data}}(\mathbf{x}) \log(D(\mathbf{x})) d\mathbf{x} + \int_{\mathbf{z}} p_{\mathbf{z}}(\mathbf{z}) \log(1 - D(g(\mathbf{z}))) d\mathbf{z}$$

$$= \int_{\mathbf{x}} \underbrace{p_{\text{data}}(\mathbf{x}) \log(D(\mathbf{x})) + p_g(\mathbf{x}) \log(1 - D(\mathbf{x}))}_{a \log(y) + b \log(1 - y)} d\mathbf{x}$$

$$a \log(y) + b \log(1 - y)$$

$$\frac{\partial}{\partial y} a \log(y) + b \log(1 - y) = 0$$

$$V(D, G) = -\log(4) + KL \left(p_{\text{data}} \left\| \frac{p_{\text{data}} + p_g}{2} \right\| \right) + KL \left(p_g \left\| \frac{p_{\text{data}} + p_g}{2} \right\| \right)$$

$$\frac{a}{y} + \frac{b}{1-y}(-1) = 0 \Rightarrow \frac{a}{y} = \frac{b}{1-y}$$

$$JSD(P||Q) = \frac{1}{2} D_{KL}(P||\frac{P+Q}{2}) + \frac{1}{2} D_{KL}(Q||\frac{P+Q}{2})$$

for $p_{\text{data}}(\mathbf{x}) =$

$$a - ay = by \Rightarrow a = ay + by = (a + b)y$$

$$\Rightarrow y = \frac{a}{a+b}$$

$$\therefore D(\mathbf{x})^* = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_g(\mathbf{x})}$$

$$D(\mathbf{x})^* = \frac{p_{\text{data}}(\mathbf{x})}{2p_{\text{data}}(\mathbf{x})} = \frac{1}{2}$$

$$V(D^*, G) = -\log(4)$$

$$V(D, G) = -\log(4) + 2 \cdot JSD(p_{\text{data}} || p_g)$$

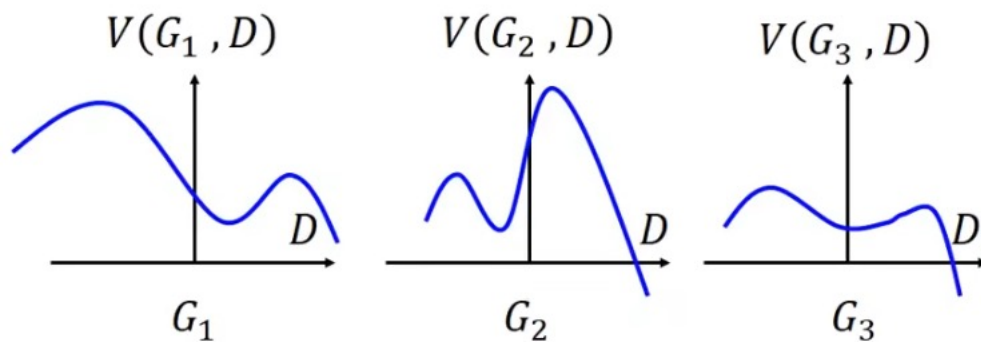


quiz

$$G^* = \arg \min_G \max_D V(G, D)$$

$$D^* = \arg \max_D V(D, G)$$

The maximum objective value is related to JS divergence.



VAE

GANs

AR

Diffusion model

Flow Based

- Variational Autoencoders (VAEs) (Kingma and Welling 2013)
- Generative Adversarial Networks (GANs) (Goodfellow *et al.* 2014)
- Flow-based Generative Models (Rezende and Mohamed 2015)
- Autoregressive Models (Vaswani *et al.* 2017)
- Diffusion Models (DDPMs) (Ho, Jain and Abbeel 2020)
- Score Based Generative Models (Song and Ermon 2019, Song *et al.* 2021)



transformer

TRANSFORMER EXPLAINER

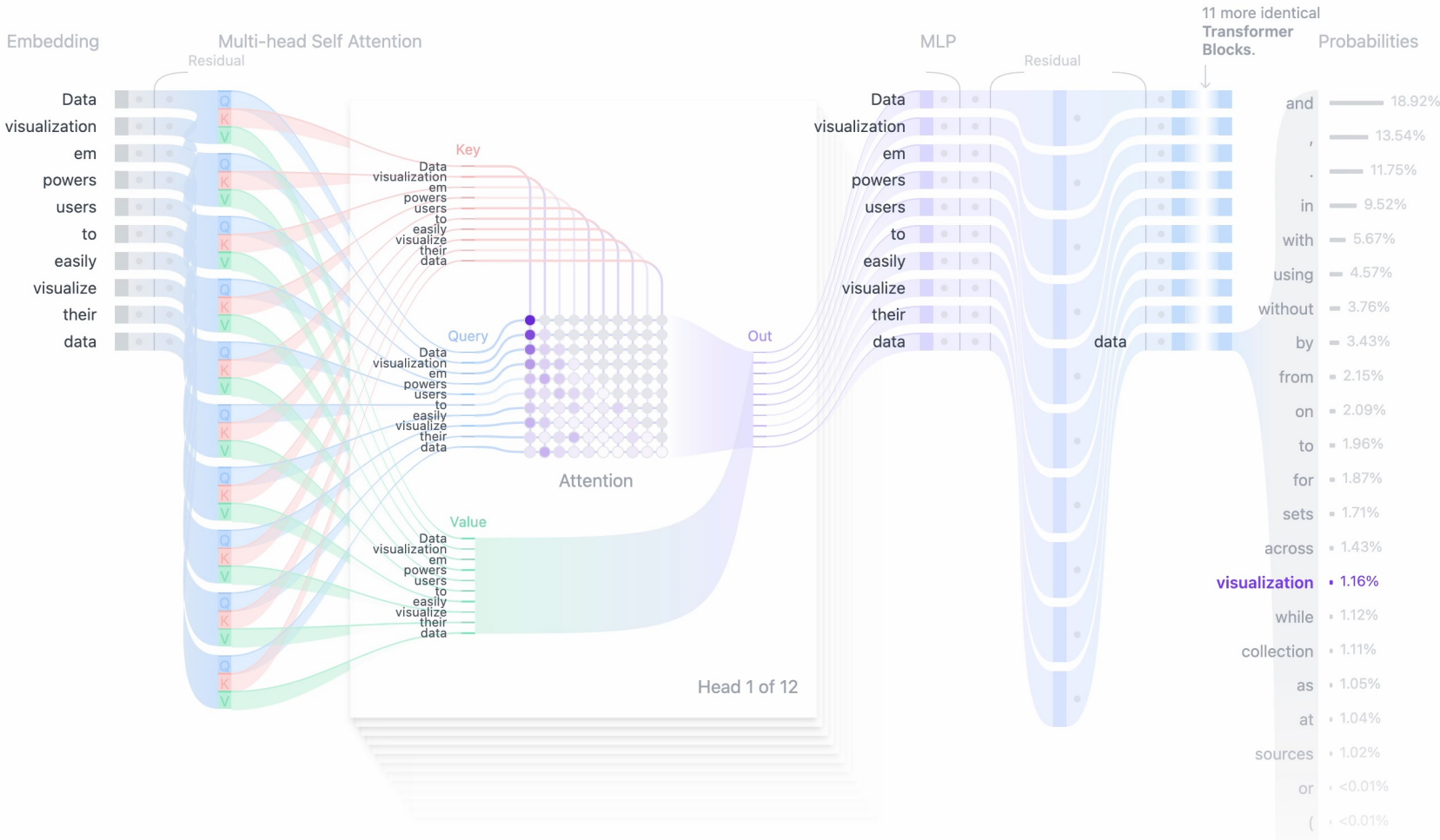
Examples ▾

Data visualization empowers users to

Generate

Temperature ?

1



VAE

GANs

AR

Diffusion Models

Flow Based

- Variational Autoencoders (VAEs) (Kingma and Welling 2013)
- Generative Adversarial Networks (GANs) (Goodfellow *et al.* 2014)
- Flow-based Generative Models (Rezende and Mohamed 2015)
- Autoregressive Models (Vaswani *et al.* 2017)
- Diffusion Models (DDPMs) (Ho, Jain and Abbeel 2020)
- Score Based Generative Models (Song and Ermon 2019, Song *et al.* 2021)



Diffusion model (DDPM)

Denoising Diffusion Probabilistic Models

Jonathan Ho
UC Berkeley
jonathanho@berkeley.edu

Ajay Jain
UC Berkeley
ajayj@berkeley.edu

Pieter Abbeel
UC Berkeley
pabbeel@cs.berkeley.edu

Abstract

We present high quality image synthesis results using diffusion probabilistic models, a class of latent variable models inspired by considerations from nonequilibrium thermodynamics. Our best results are obtained by training on a weighted variational bound designed according to a novel connection between diffusion probabilistic models and denoising score matching with Langevin dynamics, and our models naturally admit a progressive lossy decompression scheme that can be interpreted as a generalization of autoregressive decoding. On the unconditional CIFAR10 dataset, we obtain an Inception score of 9.46 and a state-of-the-art FID score of 3.17. On 256x256 LSUN, we obtain sample quality similar to ProgressiveGAN. Our implementation is available at <https://github.com/hojonathanho/diffusion>.

Diffusion model

扩散过程 (Diffusion Process) :

扩散模型的核心思想是通过逐步添加噪声来破坏分子的表示 (例如原子坐标、键关系等), 然后学习逆扩散过程, 将其逐步还原为无噪声的分子结构

前向扩散 (Forward Diffusion) :

对分子结构添加噪声, 例如通过将分子坐标逐步扰乱或修改分子图上的边连接

逆向扩散 (Reverse Diffusion) :

利用神经网络学习从噪声分布中重建无噪声分子

在逆向扩散过程中, 通常使用神经网络来预测每个时间步的噪声

训练和损失函数

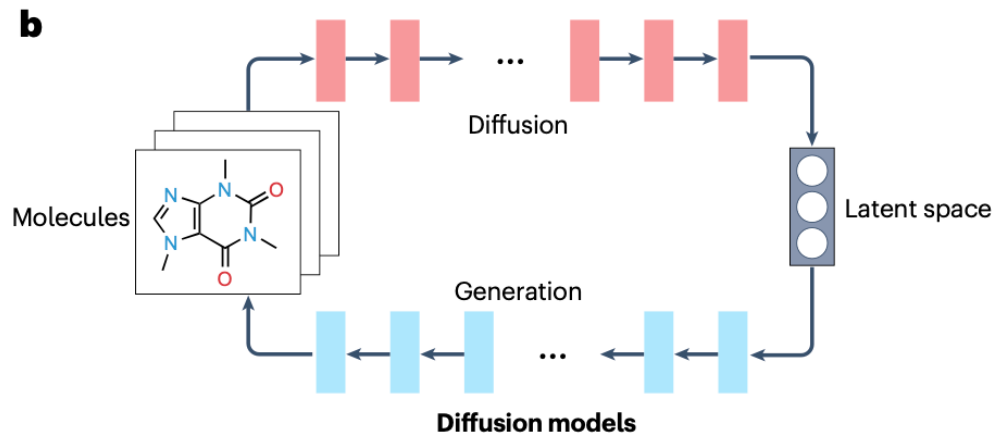
扩散模型的训练目标是最小化在每个时间步中重建无噪声分子的误差

重建损失 (Reconstruction Loss) :

衡量逆向扩散过程中, 生成的分子和原始分子之间的距离。例如, 三维坐标上的L2距离或图结构的相似性损失。

属性引导 (Property-guided Loss) :

为确保生成的分子具有特定性质, 可以引入额外的损失函数来引导模型生成具有期望性质的分子, 例如通过引入预测分子属性的辅助网络或直接优化分子性质的损失



扩散模型在分子生成中的优势

高效的样本生成:

扩散模型在生成分子时能够处理复杂的分子结构, 包括环状结构、稀有元素组合等

灵活的条件生成:

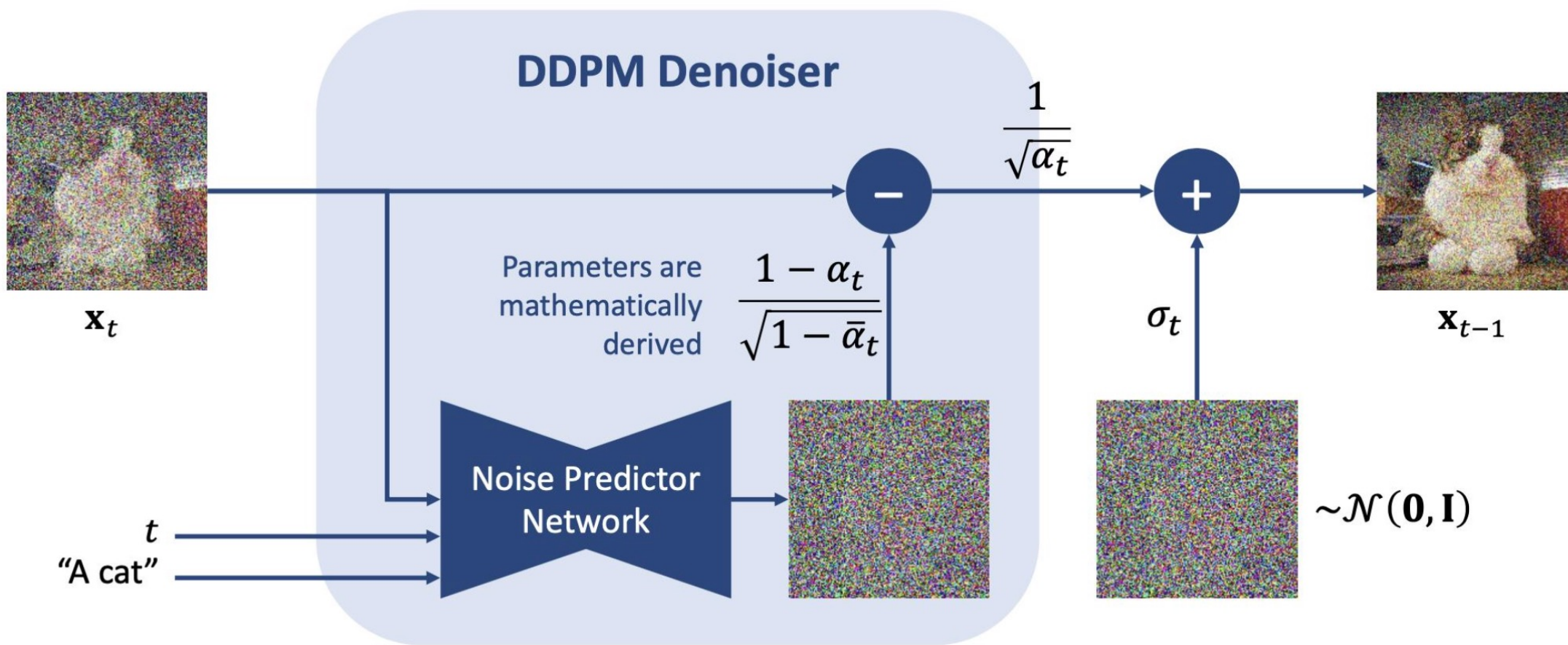
模型可以通过条件输入控制分子的物理化学性质, 生成符合目标性质的分子

高质量的生成结果:

扩散模型能够生成更符合化学规则的分子, 生成的分子通常具有较高的化学可行性和有效性

Diffusion model (DDPM)

During generation: denoise step-by-step, in each step, add noise after noise removal



diffusion model

A diffusion process : a stochastic Markov process having continuous sample paths

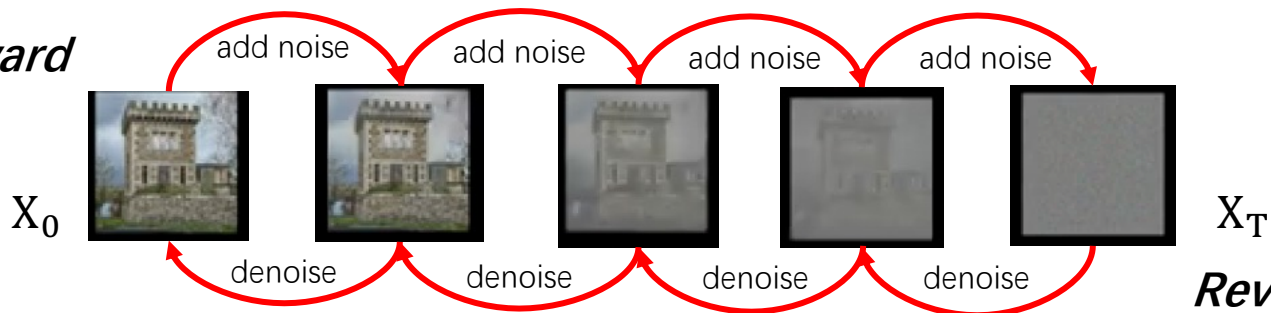
- Stochastic
- Markov process
- With no jumps

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t, \sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$$

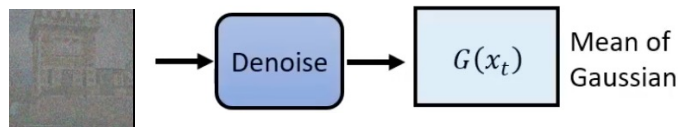
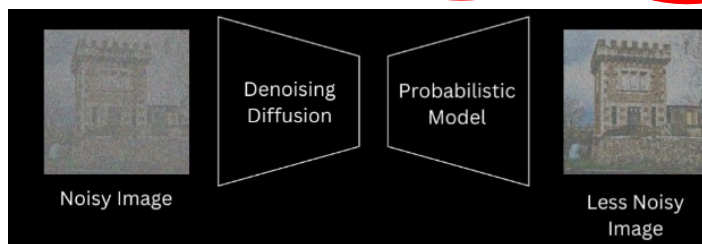
normal distribution → mean → variance → output

$$q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1})$$

Forward

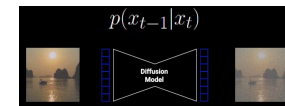


$$P(X_t|X_{t-1}, X_{t-2}, \dots, X_0) = P(X_t|X_{t-1})$$



$$P_\theta(x_{t-1}|x_t) \propto \exp(-||G(x_t) - x_{t-1}||_2)$$

$$P_\theta(x_0) = \int_{x_1:x_T} P(x_T)P_\theta(x_{T-1}|x_T) \dots P_\theta(x_{t-1}|x_t) \dots P_\theta(x_0|x_1)dx_1:x_T$$





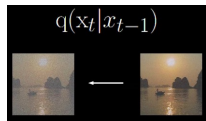
Forward diffusion process @ Diffusion model

前向过程定义：

给定从真实数据分布 $x_0 \sim q(x)$ 中采样的一个数据点 x_0 ，我们定义一个前向扩散过程，其中我们在 T 个步骤中向样本添加少量高斯噪声，生成一系列带噪声的样本 $x_1, \dots, x_T$ 。这些步骤的大小由方差调度 $\beta_t \in (0,1)$ 控制

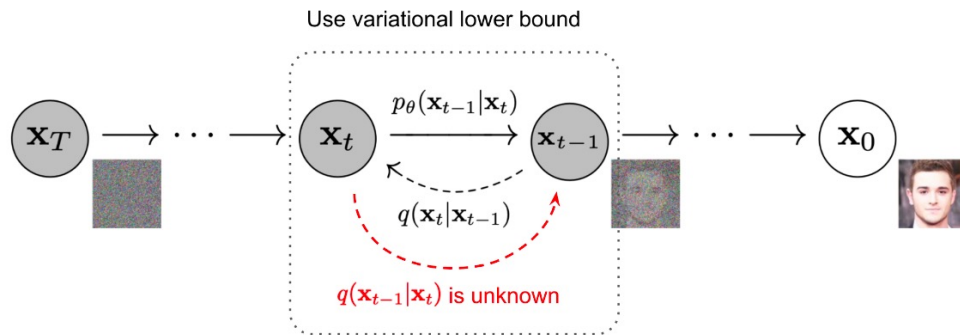
逐步模糊化的过程

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$$



前向过程的真实噪声分布

$$q(x_{1:T}|x_0) = \prod_{t=1}^T q(x_t|x_{t-1})$$



变分下界

利用变分下界来进行反向过程的推断，即如何从 x_T 回到 x_0

$p_\theta(x_{t-1}|x_t)$ 从带噪声样本中去噪的逆向过程的近似分布

$q(x_{t-1}|x_t)$ 未知的真实分布

重参数化技巧

在任意时间步 t 通过重参数化技巧对 x_t 进行采样

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon \quad \alpha_t = 1 - \beta_t \quad \bar{\alpha}_t = \prod_{i=1,t} \alpha_i \quad \epsilon \sim N(0, I)$$

高斯分布合并

将方差不同的两个高斯分布合并时，新分布的均值不变，标准差会依据公式

$$\sqrt{(1 - \alpha_t) + \alpha_t(1 - \alpha_{t-1})} = \sqrt{1 - \alpha_t\alpha_{t-1}}$$

噪声的逐步增加

噪声随时间步增加而增大，因此方差参数

$$\beta_1 < \beta_2 < \dots < \beta_T, \text{ 这使得 } \bar{\alpha}_1 > \dots > \bar{\alpha}_T$$



Langevin动力学与扩散模型

Langevin动力学

是一种基于随机微分方程的物理过程，通常用来模拟受力和随机噪声作用下的粒子运动。在Langevin动力学中，粒子在势能的引导下移动，同时受高斯噪声的影响，这使得它能够探索状态空间并避免陷入局部极小值

扩散模型

通过在一系列时间步内向数据逐步添加噪声，使数据逐渐模糊，直到接近各向同性的高斯噪声分布（前向扩散过程）。在逆向过程中，扩散模型学习如何从完全噪声的状态逐步去噪还原到原始数据

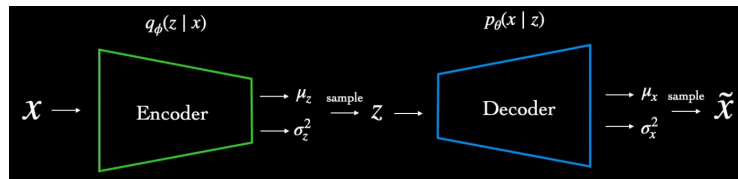
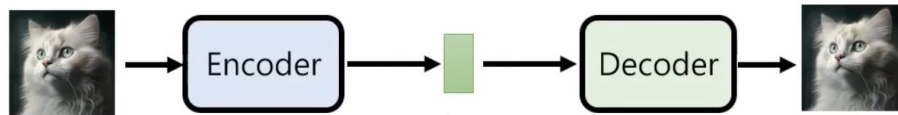
$$x_t = x_{t-1} + \frac{\delta}{2} \nabla_x \log p(x_{t-1}) + \sqrt{\delta} \epsilon_t$$

梯度 $\nabla_x \log p(x_{t-1})$ 作用是推动粒子朝着概率密度更高的区域移动

它表示了在当前状态 x_{t-1} 下，**概率密度函数 $p(x)$ 在状态空间中的变化趋势**

噪声项则增加了随机性，防止粒子陷入局部极小值

From VAE to diffusion model



$$\log p_{\theta}(x) \geq \mathbb{E}_{q(z|x)}[\log p_{\theta}(x|z)] - D_{\text{KL}}(q(z|x) \parallel p_{\theta}(z))$$

VAE Maximize $\log P_{\theta}(x)$ $\longrightarrow$ Maximize $\mathbb{E}_{q(z|x)}[\log \left(\frac{P(x, z)}{q(z|x)} \right)]$

$z \sim X_{1:T}$
 $x \sim X_0$

encoder

DDPM Maximize $\log P_{\theta}(x_0)$ $\longrightarrow$ Maximize $\mathbb{E}_{q(x_1:T|x_0)}[\log \left(\frac{P(x_0:T)}{q(x_1:T|x_0)} \right)]$

Forward process
Diffusion Process

Algorithm 1 Training

```

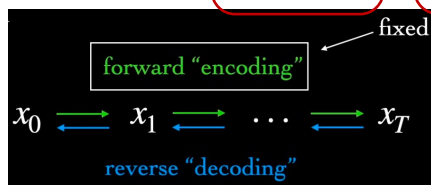
1: repeat
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$ 
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
5:   Take gradient descent step on
      $\nabla_{\theta} \left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2$ 
6: until converged
  
```

Algorithm 2 Sampling

```

1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$ 
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
5: end for
6: return  $\mathbf{x}_0$ 
  
```

$$\mathbb{E}_q \left[- \sum_{t>1} D_{\text{KL}}(q(x_{t-1}|x_t, x_0) \parallel p_{\theta}(x_{t-1}|x_t)) + \log p_{\theta}(x_0|x_1) \right] - C$$



Diffusion



$$p_{\theta}(x_{t-1}|x_t) := \mathcal{N}(x_{t-1}; \mu_{\theta}(x_t, t), \Sigma_{\theta}(x_t, t))$$

↑ learned ↑ fixed

$$\text{loss}_{\text{simple}} = \mathbb{E}_{x_0, \epsilon, t} [\| \epsilon - \epsilon_{\theta}(x_t, t) \|^2]$$



reverse diffusion process @ Diffusion model

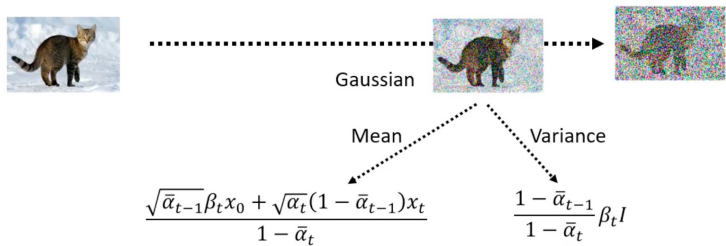
逆转前向扩散过程，并从 $q(x_{t-1}|x_t)$ 中采样，就可以通过从高斯噪声输入 $x_T \sim N(0,1)$ 开始，还原出真实数据样本。

$$L_{VLB} = \mathbb{E}_{q(\mathbf{x}_{0:T})} \left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} \right]$$

$$= \mathbb{E}_q \left[\underbrace{D_{KL}(q(\mathbf{x}_1|\mathbf{x}_0) || p_\theta(\mathbf{x}_1))}_{L_T} + \sum_{t=2}^T \underbrace{D_{KL}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) || p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}} - \log p_\theta(\mathbf{x}_0|\mathbf{x}_1) \right]_{L_0}$$

$$\mathbb{E}_{q(x_1|x_0)} [\log P(x_0|x_1)] - KL(q(x_T|x_0) || P(x_T))$$

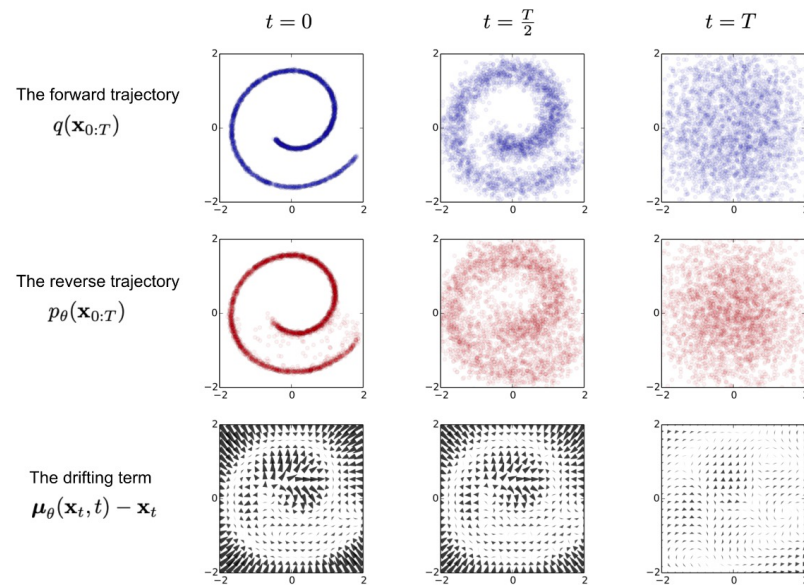
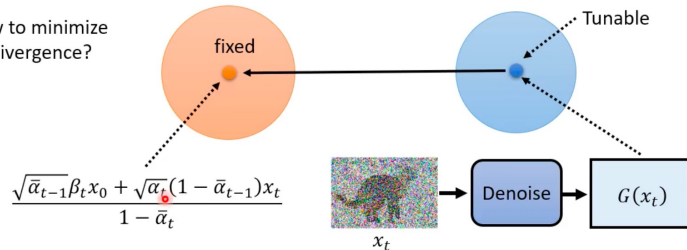
$$- \sum_{t=2}^T \mathbb{E}_{q(x_t|x_0)} [KL(q(x_{t-1}|x_t, x_0) || P(x_{t-1}|x_t))]$$



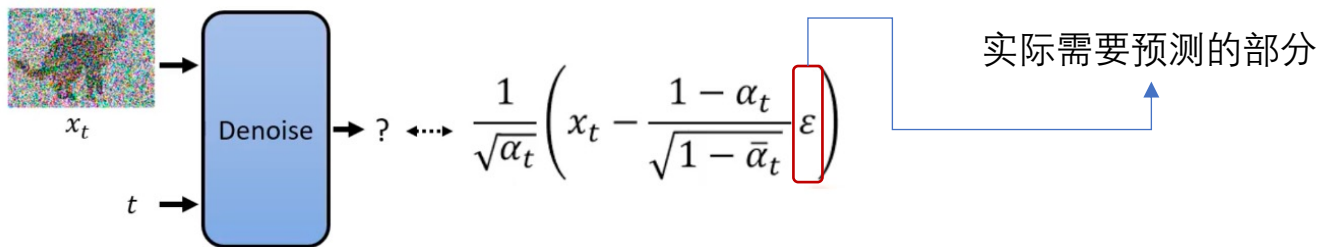
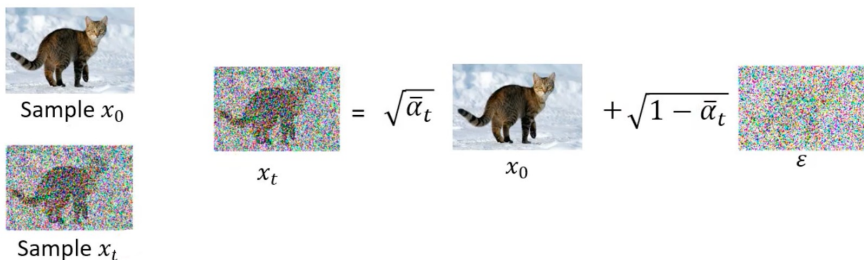
$$\mathbb{E}_{q(x_1|x_0)} [\log P(x_0|x_1)] - KL(q(x_T|x_0) || P(x_T))$$

$$- \sum_{t=2}^T \mathbb{E}_{q(x_t|x_0)} [KL(q(x_{t-1}|x_t, x_0) || P(x_{t-1}|x_t))]$$

How to minimize KL divergence?



<https://lilianweng.github.io/posts/2021-07-11-diffusion-models/>



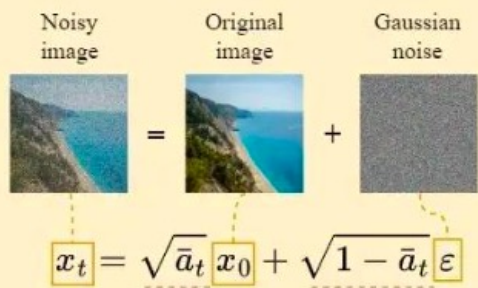
DDPM 训练流程

1. Randomly select a time step & encode it



随机选择时间步并编码

2. Add noise to image



$$\varepsilon \sim \mathcal{N}(0, 1)$$

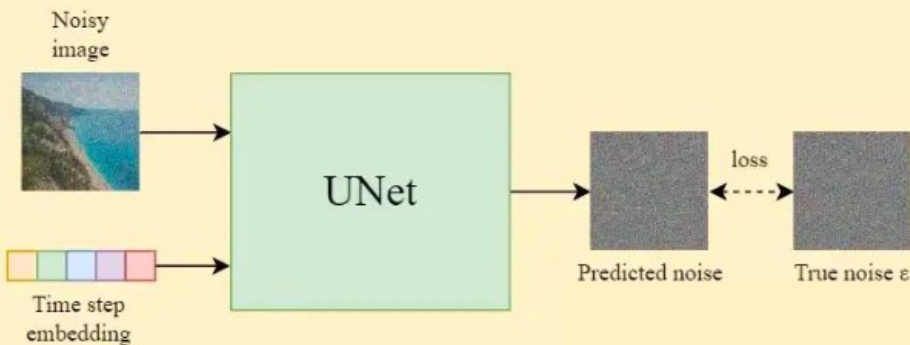
$$\alpha_t = 1 - \beta_t$$

$$\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$$

向图像添加噪声

Adjust the amount of noise according to the time step t

3. Train the UNet



训练 U-Net

DDPM 训练流程

1. Sample a Gaussian noise

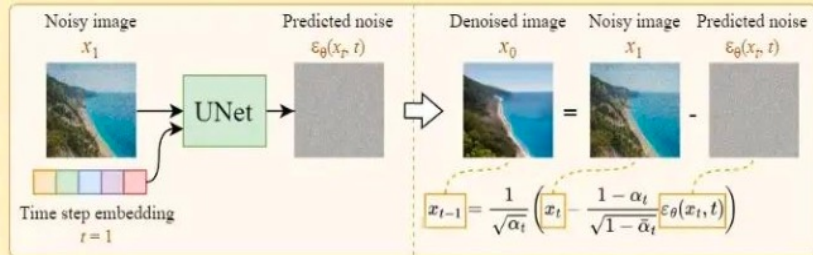
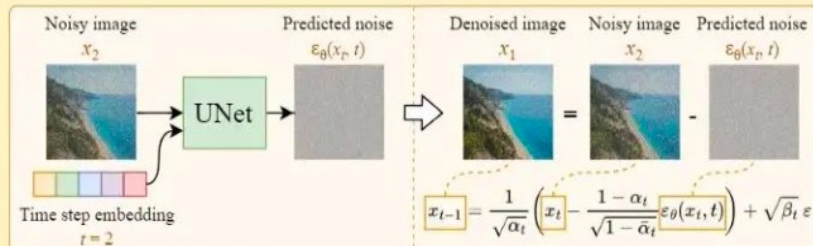
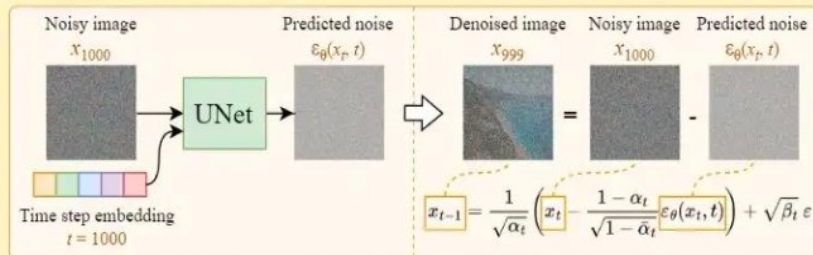
$$x_T \sim N(0, I)$$

E.g. $T = 1000$

$$x_{1000} \sim N(0, I)$$



2. Iteratively denoise the image



3. Output the denoised image

Denoised image x_0



采样高斯噪声

首先从高斯分布中采样一个噪声图像 $x_T \sim \mathcal{N}(0, I)$ ，通常选择一个较大的时间步 $T = 1000$ ，因此初始噪声图像 x_{1000} 是完全随机的噪声。

迭代去噪图像

从时间步 T 开始逐步迭代地去噪，每次减少一个时间步 t 直到达到 $t = 1$ 。在每一个时间步中：

- 将当前噪声图像 x_t 和对应的时间步嵌入输入到 UNet 模型，模型输出预测的噪声 $\epsilon_{\theta}(x_t, t)$ 。
- 通过公式 $x_{t-1} = \frac{1}{\sqrt{\alpha_t}} x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) + \sqrt{\beta_t} \epsilon$ 计算去噪后的图像 x_{t-1} ，其中：
 - α_t 和 $\bar{\alpha}_t$ 是噪声调节参数。
 - ϵ 是一个从高斯分布中采样的噪声项，用于增加少量随机性。
- 随着 t 的减小，图像逐渐清晰，逐步恢复出原始图像的结构。

这个过程从 x_{1000} 开始，依次得到 $x_{999}, x_{998}, \dots, x_1$ ，每一步都减少噪声，使得图像逐渐逼近清晰的原始图像。